DAYANANDA SAGAR COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to VTU, Approved by AICTE
Accredited by National Assessment & Accreditation Council (NAAC) with 'A' grade
Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore - 560 078.

RECORD BOOK

NAME OF THE STUDENT .....

BRANCH .....

U S N ..... SEMESTER .....

SECTION ..... CLASS ROLL No.....

SUBJECT .....

Department of Electronics & Communication Engineering

Integrated Cryptography& Network Security Lab

VII Semester (22EC73)

CAY:2026-2027



Staff in-charge:

Dr. Srividya L.

Lab Staffs:

Dr.S.Thenmozhi
Prof.Shahla Sohail
Prof.Nirmala.Y.N
Prof.Vidyashre.K.N
Prof.Aishwarya.N

Lab Instructors: Mr.Pradyumna & Mrs.Divya

HOD: Dr. Shobha K R

Name of the Student	:	
Semester /Section	:	
USN	:	
Batch	:	

Dayananda Sagar College of Engineering

Shavige Malleshwara Hills, Kumaraswamy Layout,
Banashankari, Bangalore-560111, Karnataka
Tel : +91 80 26662226 26661104 Extn : 2731 Fax : +90 80 2666 0789
Web - http://www.dayanandasagar.edu Email : hod-ece@dayanandasagar.edu
(An Autonomous Institute Affiliated to VTU, Approved by AICTE)

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU. Approved by AICTE)
Accredited by National Assessment & Accreditation Council (NAAC) with 'A' grade
Shavige Maileshwara Hills, Kumaraswamy Layout, Bangalore - 580.078.

Department
of
Electronics and Communication Engineering

Integrated Cryptography & Network Security Laboratory
Code: 22EC73

Laboratory Certificate

This is to certify that Mr./Ms. ..... *bearing USN* ..... *has satisfactorily completed the experiments in above practical subject prescribed by the* **Visvesvaraya Technological University** *for the* **'VII'** *Semester B.E. in the* aforesaid **ICNS laboratory** *during the year 2026-2027*

Date :

Marks	
Maximum	**Obtained**

Name of the Faculty

Signature of Faculty Incharge

Head of the Department

About the College

The Dayananda Sagar College of Engineering was established in 1979 by Sri R. Dayananda Sagar. Dayananda Sagar College of Engineering operates under the aegis of the **Mahatma Gandhi Vidya Peetha Trust** is approved by All India Council for Technical Education (AICTE), Govt. of India and affiliated to Visvesvaraya Technological University. It has the widest choice of engineering branches having 18 Under Graduate courses & 6 Post Graduate courses. In addition, it has 20 Research Centres in different branches of Engineering catering to research scholars for obtaining Ph.D under VTU. Various courses are accredited by NBA.

The Institute is spread over 23 acres of land with large infrastructure supported by laboratories with state-of-the-art, Equipment & Machines. The Central Library with modern facilities and the Digital Library provides the knowledge base for the students.

The campus is WI-FI equipped with large bandwidth internet facility. The College has good faculty strength with highest professional integrity and committed to the academics with transparency in their actions. Each faculty takes the responsibility of mentoring a certain number of students' through personal attention paving the way for the students' professional growth. The faculty are research oriented having number of sponsored R & D projects from different agencies such as Department of Science & Technology, Defense R & D organizations, Indian Space Research Organization, AICTE etc..

About the Department

Department of Electronics and Communication Engineering is one of the oldest and biggest department in the DSCE group with 52 faculty members and 1200+ students. Most of our faculty have pursued Ph.D. and others are in the process. The ECE department provides high-quality, innovative technical education at the undergraduate, graduate, and research levels. At present, the department offers an UG course (BE) with an intake of 360, a PG course in VLSI Design and Embedded Systems with an intake of 18 students and Ph.D. The department has got an excellent infrastructure with sophisticated labs, class rooms and R & D center. The department faculty and support personnel are dedicated towards helping students achieve success in today's world by offering them the knowledge and skills required. The department places a strong emphasis on leading-edge research through its research centre and research forum. These factors together enable our graduating students to create a big impact on the workforce from day one.

In addition to offering innovative courses, the department is more focused on improving students' educational experiences by offering value-added courses, skill-development programs, technical challenges and hackathons at state, national, and international arenas. Through its professional societies and technical clubs, the department takes great satisfaction in exposing its students to current industry developments and upcoming technologies. The dedicated staff at the ECE department help students procure jobs in prestigious companies both domestically and abroad.

Vision of the College:

To impart quality technical education with a focus on Research and Innovation emphasizing on Development of Sustainable and Inclusive Technology for the benefit of society.

Mission of the College:

- To provide an environment that enhances creativity and Innovation in pursuit of Excellence.
- To nurture teamwork in order to transform individuals as responsible leaders and entrepreneurs.
- To train the students to the changing technical scenario and make them to understand the importance of Sustainable and Inclusive technologies.

Vision of the Department

To achieve continuous improvement in quality technical education for global competence with focuson industry, societal needs, research and professional success.

Mission of the Department

- Offering quality education in Electronics and Communication Engineering with effective teaching learning process in multidisciplinary environment.
- Training the students to take-up projects in emerging technologies and work with team spirit.
- To imbibe professional ethics, development of skills and research culture for better placement opportunities.

Program Educational Objectives [PEOs]

The Graduate students must be able to:

PEO-1: Ready to apply the state-of-art technology in industry and meeting the societal needs with

Knowledge of Electronics and Communication Engineering due to strong academic culture.

PEO-2: Competent in technical and soft skills to be employed with capability of working in

Multidisciplinary domains.

PEO-3: Professionals, capable of pursuing higher studies in technical, research or management programs.

Programme Specific Outcomes [PSOs]

PSO-1: Design, develop and integrate electronic circuits and systems using current practices and Standards.
PSO-2: Apply knowledge of hardware and software in designing Embedded and Communication Systems.

Knowledge and Attitude Profile (WK) as per New NBA-2024

WK1: A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.

WK2: Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.

WK3: A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.

WK4: Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.

WK5: Knowledge, including efficient resource use, environmental impacts, whole-life cost, reuse of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.

WK6: Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.

WK7: Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.

WK8: Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.

WK9: Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

New POs as per NBA-2024

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

SDGs (Sustainable Development Goals) from UN

1. **No Poverty**: End poverty in all its forms everywhere
2. **Zero Hunger:** End hunger, achieve food security and improved nutrition and promote sustainable agriculture
3. **Good Health and Wellbeing:** Ensure healthy lives and promote well-being for all at all ages
4. **Quality Education:** Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all
5. **Gender equality:** Achieve gender equality and empower all women and girls
6. **Clean water and Sanitation:** Ensure availability and sustainable management of water and sanitation for all
7. **Affordable and Clean Energy:** Ensure access to affordable, reliable, sustainable and modern energy for all
8. **Decent work and Economic growth:** Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all
9. **Industry, Innovation and Infrastructure:** Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation
10. **Reduced Inequalities:** Reduce inequality within and among countries
11. **Sustainable cities and Communities:** Make cities and human settlements inclusive, safe, resilient and sustainable
12. **Responsible consumption and production:** Ensure sustainable consumption and production patterns
13. **Climate action:** Take urgent action to combat climate change and its impacts
14. **Life below water:** Conserve and sustainably use the oceans, seas and marine resources for sustainable development
15. **Life on Land:** Protect, restore and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, halt and reverse land degradation, and halt biodiversity loss
16. **Peace, justice and strong institutions:** Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels
17. **Partnership for Goals:** Strengthen the means of implementation and revitalize the global partnership for sustainable development

INTEGRATED CRYPTOGRAPHY& NETWORK SECURITY LAB (IPCC)								
Course Code	:	22EC73				**Credits**	:	3(theory)+1(lab)
L: T: P	:	3	1	1		**CIE Marks**	:	50
Exam Hours	:	3				**SEE Marks**	:	50
Total Hours	:	24				**CIE + SEE Marks**	:	100

Course Objectives:

- To understand basic fundamentals of Cryptography and Network Security.
- To acquire knowledge on standard algorithms used to provide confidentiality, integrity, and authenticity.
- To understand the various key distribution and management schemes.
- To understand various protocols for network security to protect against threats in the networks.

Module	Contents	Hours
1	The OSI security architecture, Security Attacks, Security Services, Security Mechanisms, Network security model, and Cryptographic Standards. Classical Encryption techniques (Symmetric cipher model, substitution techniques-Caesar, Playfair, Hill ciphers, transposition techniques). **Block Ciphers: The Feistel Cipher,** Data Encryption Standard (DES)-Block cipher Design principles. Block Cipher Modes of Operation	08
2	**Block Ciphers & Public Key Cryptography** **Block Ciphers: Advanced** Encryption Standard (AES). Blowfish. **Public Key Cryptography:** The RSA algorithm-Key management - Diffie Hellman Key Exchange-Elliptic curve arithmetic-Elliptic curve cryptography.	08
3	**Hash Functions and Digital Signatures:** **Hash Functions:** Applications, Requirements and Security, SHA. **Message Authentication Codes**: Message authentication Requirements, Message authentication Functions, Requirements for MAC's, MAC's based on Hash functions. **Digital Signature:** Digital signatures Requirements, DSA.	08
4	**System Security** **Intruders:** - Intrusion detection system, password management **Malicious software: -**Virus and related threats, Virus Counter measures **Fire walls:-** Firewall characteristics, Types of Firewalls, Firewall configurations, Concept of trusted system	08
5	**Internet Security:** **E-mail Security:** Internet Mail architecture, E-mail formats, E-mail threats and security, S/MIME **IP Security:** Overview of IP Security , IP Security policy, Encapsulating Security Payload **Cloud Security:** Cloud computing, Cloud security concepts, Cloud security risks and countermeasures, Cloud security as a service.	08

Self-study components

Module 1 Introduction to Number Theory (Full chapter 2 from 8th edition)
Module 2 BLOCK CIPHERS - Finite Fields(Chapter 5 from 8th edition), Triple DES- RC5 algorithm
Module 3 Hash Functions and Digital Signatures Hash Functions- HMAC EI Gamal – Schnorr.

Module 4 Security Practice & System Security - Firewall designs - SET for E-Commerce Transactions.
Module 5 Internet Security: client authentication-PKI as deployed by SSL Attacks fixed in v3-Exportability

Textbooks:

1 William Stallings, "Cryptography and Network Security: Principles and Practice", *Prentice Hall*, 8th edition.

Reference Books:

1 Atul Kahate, "Cryptography and Network Security", 3rd edition.

2 Behrouz A. Forouzan, "Data Communications and Networking", 2nd edition.

3 J.W. Rittiaghouse and William M. Hancok, "Cyber Security Operations Handbook", *Elsevier*.

4 A.J. Menezes, P. Van Oorschot and S. Vanstone, "Handbook of Applied Cryptography", *CRC Press*, 1997, ISBN: 0-84-938523-7.

List of Experiments in IPCC Course
Integrated Lab

Simulation experiments using software tools:

Expt. No.	Contents of the Experiments	Hrs
1	Perform encryption and decryption using the following Substitution cipher algorithms: a) Ceaser Cipher b) Monoalphabetic Cipher c) Hill Cipher	**2**
2	Perform encryption and decryption using the following transposition techniques i. Rail fence ii. Row & Column Transformation	**2**
3	Implement a simple Data Encryption Standard algorithm	**2**
4	Implement a simplified Advanced Encryption Standard algorithm	**2**
5	Implement Secure Online Transactions (like e-commerce or banking) using the RSA algorithm	**2**
6	Implement Diffie-Hellman Key Exchange to establish session keys in a mobile application	**2**
7	Calculate the message digest of a text using the SHA-1 algorithm.	**2**
8	Secure Bitcoin Transaction Signing with Elliptic Curve Cryptography.	**2**
9	Implementing Digital Signatures for Secure E-Governance Services (Tax Filing, Aadhaar e-KYC, and E-Voting)	**2**
10	a. Build a Trojan and know the impact of the Trojan malware on a computer system b. Install a rootkit hunter and find the malware in a computer.	**2**

11	Induce and combat an attack by an intrusion detection system(IDS) using any network software tools	**2**
12	Evaluate the vulnerability of a network by automated attack and Penetration Tools monitoring tools	**2**

Integrated Laboratory Experiment Reference Textbooks:

1 William Stallings, "Cryptography and Network Security: Principles and Practice", *Prentice Hall*, 8th edition.

On-Line Materials & Resources:

1 nptel.ac.in/courses/106106221
2 https://youtube.com/playlist?list=PLO0SXQmz3ypmofNFSPnRR7JbJmwvJPSWM&si=SMpH3yJV_IVNE6y1
3 https://youtu.be/GfSPxH-tkug?si=fPxbWlqmvpKH_ZO7

Course Outcomes:

After completion of the course, the graduates will be able to

CO1 Understand the principles and techniques of symmetric and asymmetric encryption, including block ciphers and key distribution methods.
CO2 Apply standard cryptographic algorithms like RSA,DES,AES,Hash functions and digital signatures to ensure confidentiality , integrity, and authenticity in communication.
CO3 Analyse the security strengths, vulnerabilities, and computational feasibility of cryptographic techniques.
CO4 Demonstrate the application of cryptographic techniques in securing systems, emails, IP communications, and cloud-based applications.
CO5 Apply cryptographic and security mechanisms to develop secure communication systems that address sustainable development goals in health, governance, and digital infrastructure.

DAYANANDA SAGAR COLLEGE OF ENGINEERING

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

DO's

- All the students should come to LAB on time with proper dress code and identity cards.
- Keep your belongings in the bag rack of laboratory.
- Students have to enter their name, USN, time-in/out and signature in the log register maintained in the laboratory.
- All the students should submit their records before the commencement of Laboratory experiments.
- Students should come to the lab well prepared for the experiments which are to be performed in that particular session.
- Students are asked to do the experiments on their own and should not waste their precious time by talking, roaming and sitting idle in the labs.
- Observation book and record book should be complete in all respects and it should be corrected by the staff member.
- Before leaving the laboratory students should arrange their chairs and leave in orderly manner after completion of their scheduled time.
- Prior permission to be taken, if for some reasons, they cannot attend lab.
- Immediately report any sparks/ accidents/ injuries/ any other untoward incident to the faculty /instructor.
- In case of an emergency or accident, follow the safety procedure.

Switch OFF the power supply after completion of experiment

DONT's

- The use of mobile/ any other personal electronic gadgets is prohibited in the laboratory.
- Do not make noise in the Laboratory & do not sit on experiment table.
- Do not make loose connections and avoid overlapping of wires
- Don't switch on power supply without prior permission from the concerned staff.
- Never leave the experiments while in progress.
- Do not leave the Laboratory without the signature of the concerned staff in observation book.

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

INTEGRATED CRYPTOGRAPHYAND NETWORK SECURITY LAB
22EC73

Index Sheet

Sl.No	Program	Date of conduction	Marks obtained					Sign
			C (10)	A (10)	R (5)	V (5)	Total	
1	Perform encryption and decryption using following Substitution cipher algorithms: a) Caesar Cipher b) Mono alphabetic Cipher c) Hill Cipher							
2	Perform encryption and decryption using following transposition techniques i. Rail fence ii. Row & Column Transformation							
3	Implement a simple Data Encryption Standard algorithm							
4	Implement a simplified Advanced Encryption Standard algorithm							
5	Implement Secure Online Transactions (like e-commerce or banking) using the RSA algorithm							
6	Implement Diffie-Hellman Key Exchange to establish session keys in a mobile application							
7	Calculate the message digest of a text using the SHA-1 algorithm.							
8	Secure Bitcoin Transaction Signing with Elliptic Curve Cryptography.							
9	Implementing Digital Signatures for Secure E-Governance Services (Tax Filing, Aadhaar e-KYC, and E- Voting)							
10	a. Build a Trojan and know the impact of the Trojan malwares in a computer system b. Install a rootkit hunter and find the malwares in a computer							
11.	Induce and combat an attack by intrusion detection system (IDS) using any network software tools.							
12.	Evaluate vulnerability of a network by automated Attack and Penetration Tools							

INTEGRATED CRYPTOGRAPHY AND NETWORK SECURITY LAB

CONTENTS

Expt. No.	Contents of the Experiments	PAGE No.
1	Perform encryption and decryption using following Substitution cipher algorithms: a) Caesar Cipher b) Mono alphabetic Cipher c) Hill Cipher	1
2	Perform encryption and decryption using following transposition techniques i. Rail fence ii. Row & Column Transformation	8
3	Implement a simple Data Encryption Standard algorithm	13
4	Implement a simplified Advanced Encryption Standard algorithm	17
5	Implement Secure Online Transactions (like e-commerce or banking) using the RSA algorithm	22
6	Implement Diffie-Hellman Key Exchange to establish session keys in a mobile application	26
7	Calculate the message digest of a text using the SHA-1 algorithm.	29
8.	Secure Bitcoin Transaction Signing with Elliptic Curve Cryptography.	32
9	Implementing Digital Signatures for Secure E-Governance Services (Tax Filing, Aadhaar e-KYC, and E-Voting)	36
10	a. Build a Trojan and know the impact of Trojan malware in a computer system b. Install a rootkit hunter and find the malware in a computer	40
11	Induce and combat an attack by intrusion detection system (IDS) using any network Software tools.	44
12	Evaluate the vulnerability of a network by automated Attack and Penetration Tools	48

Date:

Program-1: To implement the following Substitution cipher algorithms:

a) **Ceaser Cipher**

b) **Mono alphabetic Cipher**

c) **Hill Cipher**

1a. Ceaser Cipher

The earliest known, and the simplest, use of a substitution cipher was by Julius Caesar. The Caesar cipher involves replacing each letter of the alphabet with the letter standing three places further down the alphabet. For example,

```
plain:  meet me after the toga party
cipher: PHHW PH DIWHU WKH WRJD SDUWB
```

Note that the alphabet is wrapped around, so that the letter following Z is A. We can define the transformation by listing all possibilities, as follows:

```
plain:   a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher:  D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```

Let us assign a numerical equivalent to each letter:

a	b	c	d	e	f	g	h	i	j	k	l	m
0	1	2	3	4	5	6	7	8	9	10	11	12

n	o	p	q	r	s	t	u	v	w	x	y	z
13	14	15	16	17	18	19	20	21	22	23	24	25

Then the algorithm can be expressed as follows. For each plaintext letter, substitute the cipher text letter C:

$$C = \mathrm{E}(3, p) = (p + 3) \bmod 26$$

A shift may be of any amount, so that the general Caesar algorithm is:

$$C = \mathrm{E}(k, p) = (p + k) \bmod 26$$

Where takes on a value in the range 1 to 25. The decryption algorithm is simply

$$p = \mathrm{D}(k, C) = (C - k) \bmod 26$$

MATLAB CODE:

```
% Caesar Cipher in MATLAB
str = input('Enter any string: ', 's');
key = input('Enter the Key: ');
```

```
    encrypted = encrypt(str, key);
    fprintf('\nEncrypted String is: %s\n', encrypted);

    decrypted = decrypt(encrypted, key);
    fprintf('Decrypted String is: %s\n', decrypted);
```

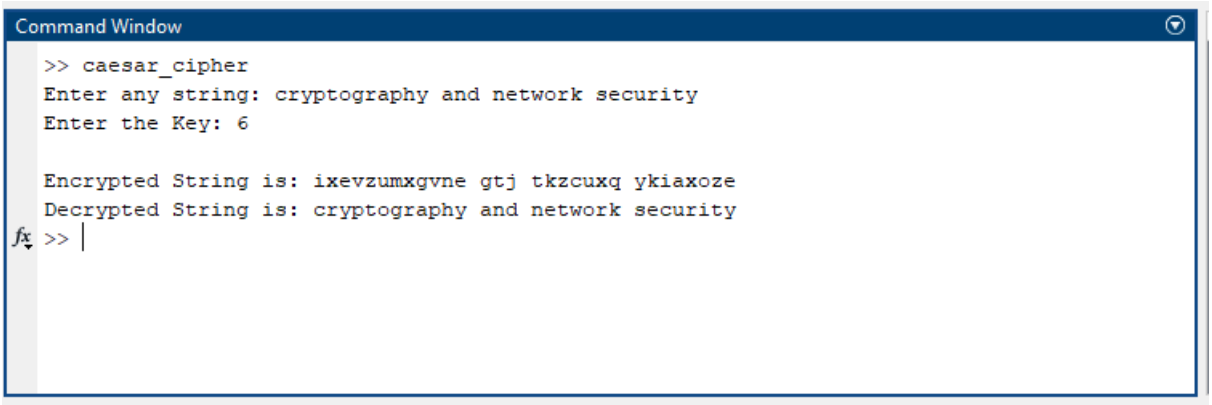
```
function encrypted = encrypt(str, key)
    encrypted = '';
    key = mod(key, 26);

    for i = 1:length(str)
        ch = str(i);
        if isstrprop(ch, 'upper')
            c = double(ch) + key;
            if c > double('Z')
                c = c - 26;
            end
        elseif isstrprop(ch, 'lower')
            c = double(ch) + key;
            if c > double('z')
                c = c - 26;
            end
        else
            c = double(ch); % non-alphabetic characters remain unchanged
        end
        encrypted(end + 1) = char(c);
    end
end
```

```
function decrypted = decrypt(str, key)
    decrypted = '';
    key = mod(key, 26);

    for i = 1:length(str)
        ch = str(i);
        if isstrprop(ch, 'upper')
            c = double(ch) - key;
            if c < double('A')
                c = c + 26;
            end
        elseif isstrprop(ch, 'lower')
            c = double(ch) - key;
            if c < double('a')
                c = c + 26;
            end
        else
            c = double(ch); % non-alphabetic characters remain unchanged
        end
        decrypted(end + 1) = char(c);
    end
end
```

OUTPUT:



b) Monoalphabetic Cipher

If, instead, the "cipher" line can be any permutation of the 26 alphabetic characters, then there are 26! or greater than possible keys. This is 10 orders of magnitude greater than the key space for DES and would seem to eliminate brute-force techniques for cryptanalysis. Such an approach is referred to as a monoalphabetic substitution cipher, because a single cipher alphabet (mapping from plain alphabet to cipher alphabet) is used per message.

```
    % Substitution Cipher in MATLAB (Encrypt → Decrypt Automatically)

a = 'abcdefghijklmnopqrstuvwxyz';
b = "@%&wvutsrq24nmlkjihg97dcba";

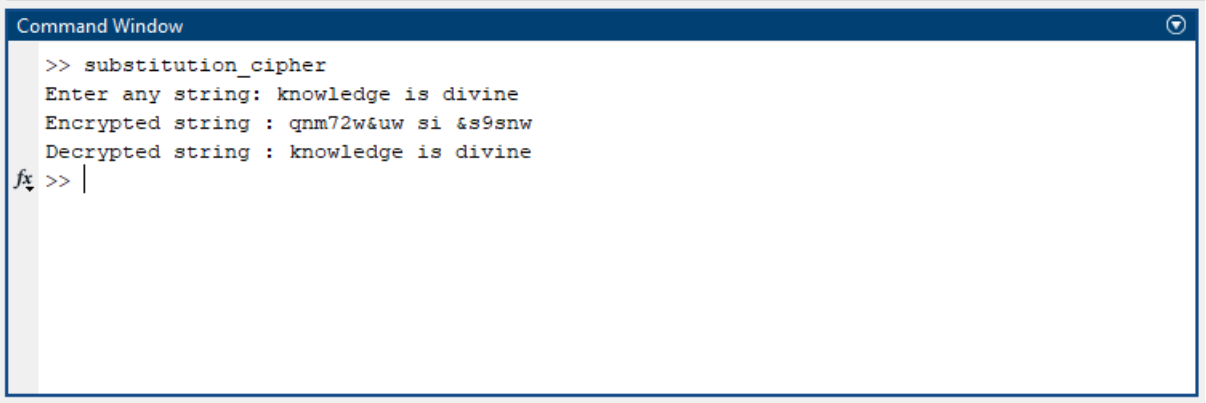
inputStr = input('Enter any string: ', 's');
inputStr = lower(inputStr);  % Convert to lowercase
encrypted = '';
decrypted = '';

% --------- Encryption ---------
for i = 1:length(inputStr)
   c = inputStr(i);
   idx = strfind(a, c);
   if ~isempty(idx)
      encrypted(end+1) = b(idx);  % Encrypt letter
   else
      encrypted(end+1) = c;      % Preserve non-letters
   end
end

% --------- Decryption ---------
for i = 1:length(encrypted)
```

```
    c = encrypted(i);
    idx = strfind(b, c);
    if ~isempty(idx)
       decrypted(end+1) = a(idx);  % Decrypt character
    else
       decrypted(end+1) = c;       % Preserve non-cipher characters
    end
end

% --------- Display results ---------
fprintf('Encrypted string : %s\n', encrypted);
fprintf('Decrypted string : %s\n', decrypted);
```



c) Hill Cipher

THE HILL ALGORITHM This encryption algorithm takes *m* successive plaintext letters and substitutes for them m ciphertext letters. The substitution is determined m by linear equations in which each character is assigned a numerical value. For m=3 , the system can be described as:

$$c_1 = (k_{11}p_1 + k_{12}p_2 + k_{13}p_3) \bmod 26$$
$$c_2 = (k_{21}p_1 + k_{22}p_2 + k_{23}p_3) \bmod 26$$
$$c_3 = (k_{31}p_1 + k_{32}p_2 + k_{33}p_3) \bmod 26$$

This can be expressed in terms of row vectors and matrices:

$$(c_1\ c_2\ c_3) = (p\ \ p_2\ p_3)\begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix} \bmod 26$$

OR

$$\mathbf{C} = \mathbf{PK} \bmod 26$$

where C and P are row vectors of length 3 representing the plaintext and cipher text, and K is a 3x3 matrix representing the encryption key. Operations are performed mod 26.

EXAMPLE:

consider the plaintext “paymoremoney” and use the encryption key

$$\mathbf{K} = \begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix}$$

The first three letters of the plaintext are represented by the vector (15 0 24). Then (15 0 24)$\mathbf{K}$ = (303 303 531) mod 26 = (17 17 11) = RRL. Continuing in this fashion, the ciphertext for the entire plaintext is RRLMWBKASPDH.

Decryption requires using the inverse of the matrix $\mathbf{K}$. We can compute $\det \mathbf{K} = 23$, and therefore, $(\det \mathbf{K})^{-1} \bmod 26 = 17$. We can then compute the inverse as

$$\mathbf{K}^{-1} = \begin{pmatrix} 4 & 9 & 15 \\ 15 & 17 & 6 \\ 24 & 0 & 17 \end{pmatrix}$$

This is demonstrated as

$$\begin{pmatrix} 17 & 17 & 5 \\ 21 & 18 & 21 \\ 2 & 2 & 19 \end{pmatrix} \begin{pmatrix} 4 & 9 & 15 \\ 15 & 17 & 6 \\ 24 & 0 & 17 \end{pmatrix} = \begin{pmatrix} 443 & 442 & 442 \\ 858 & 495 & 780 \\ 494 & 52 & 365 \end{pmatrix} \bmod 26 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

It is easily seen that if the matrix $\mathbf{K}^{-1}$ is applied to the ciphertext, then the plaintext is recovered.

In general terms, the Hill system can be expressed as

$$\mathbf{C} = E(\mathbf{K}, \mathbf{P}) = \mathbf{PK} \bmod 26$$
$$\mathbf{P} = D(\mathbf{K}, \mathbf{C}) = \mathbf{CK}^{-1} \bmod 26 = \mathbf{PKK}^{-1} = \mathbf{P}$$

MATLAB CODE:

```
function hill_cipher()
    % HILL CIPHER ENCRYPTION & DECRYPTION (3x3 key, 3-letter message)

    % Get key matrix (3x3)
    fprintf('Enter 3x3 key matrix (it should be invertible mod 26):\n');
    key = zeros(3,3);
    for i = 1:3
        for j = 1:3
            key(i,j) = input(sprintf('Enter element (%d,%d): ', i, j));
        end
    end
    % Get message
    msg = input('Enter a 3-letter lowercase message: ', 's');
    if length(msg) ~= 3
        error('Message must be exactly 3 characters.');
    end
    msg_vec = double(msg) - 97; % convert chars to numbers (0-25)

    % Encrypt
    encrypted_vec = mod(key * msg_vec', 26);
    encrypted_msg = char(encrypted_vec' + 97); % convert back to characters
    fprintf('\nEncrypted string is: %s\n', encrypted_msg);

    % Inverse key modulo 26
    inv_key = inverse_mod26(key);
```

```
    % Decrypt
    decrypted_vec = mod(inv_key * encrypted_vec, 26);
    decrypted_msg = char(decrypted_vec' + 97);
    fprintf('Decrypted string is: %s\n', decrypted_msg);
end
```

```
function inv = modinv(a, m)
  % Modular inverse using extended Euclidean algorithm
  [g, x, ~] = gcd_extended(a, m);
  if g ~= 1
    error('Modular inverse does not exist');
  else
    inv = mod(x, m);
  end
end
```

```
function invA = inverse_mod26(A)
  % Compute inverse of matrix A under mod 26 arithmetic
  detA = round(det(A));
  detA = mod(detA, 26);

  if gcd(detA, 26) ~= 1
    error('Key matrix is not invertible modulo
  26'); end

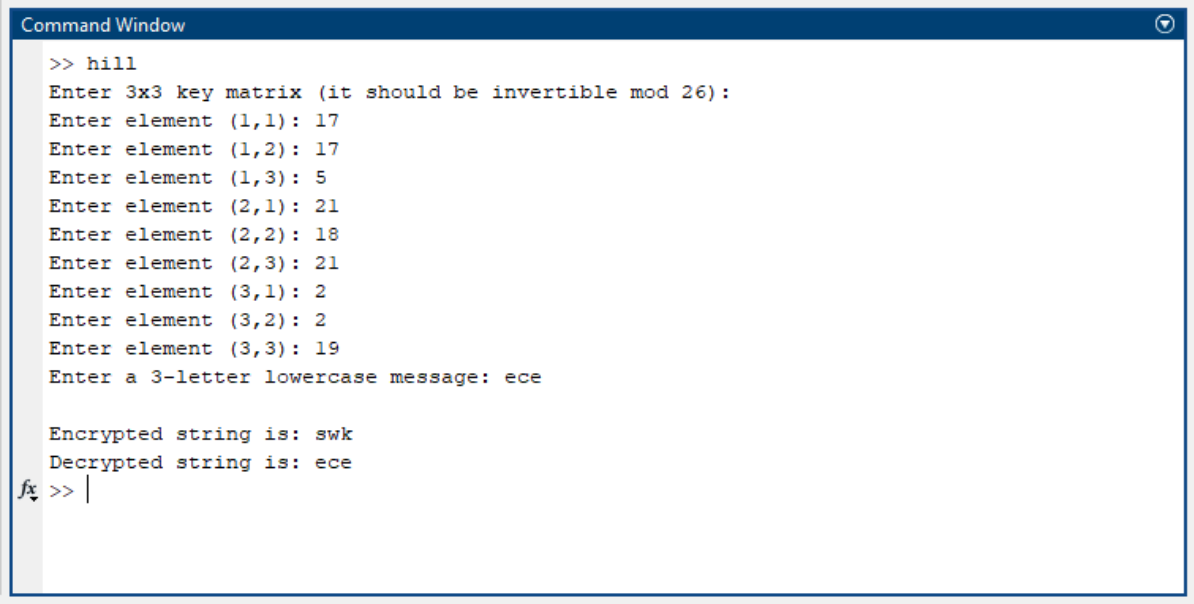
  det_inv = modinv(detA, 26); % modular inverse of determinant

  % Cofactor matrix
  cofactor = zeros(3,3);
  for i = 1:3
    for j = 1:3
      minor = A;
      minor(i,:) =
      [];
      minor(:,j) = [];
      cofactor(i,j) = (-1)^(i+j) * round(det(minor));
    end
  end

  adjugate = cofactor'; % transpose of cofactor
  invA = mod(det_inv * adjugate, 26);
```

```
function [g, x, y] = gcd_extended(a, b)
  % Extended Euclidean Algorithm
  if b == 0
    g = a;
    x = 1;
    y = 0;
  else
```

```
    [g, x1, y1] = gcd_extended(b,
    mod(a,b)); x = y1;
    y = x1 - floor(a/b)*y1;
  end
end
```



Assignment:

1. **Improve the strength of the given cipher either by increasing number of rounds or by combining two or more algorithms.**
2. **Write a MATLAB code to give user a choice to select desired cipher**

Signature:

Date:

Program-2: To implement a program for encryption and decryption using rail fence transposition technique and the row and column transformation technique.

2a.Rail Fence Algorithm

ALGORITHM:

1. In the rail fence cipher, the plaintext is written downwards and diagonally on successive "rails" of an imaginary fence, then moving up when we reach the bottom rail.
2. When we reach the top rail, the message is written downwards again until the whole plaintext is written out.
3. The message is then read off in rows.

MATLAB CODE:

```
clc;
clear all;
% Rail Fence Cipher in MATLAB
msg = 'dayananda';
depth = 3;

enc = encode(msg, depth);
dec = decode(enc, depth);

fprintf('Simulating Railfence Cipher\n');
fprintf('--------------------------\n');
fprintf('Input Message     : %s\n', msg);
fprintf('Encrypted Message : %s\n', enc);
fprintf('Decrypted Message : %s\n', dec);

% ---------------- Encryption ----------------
function enc = encode(msg, depth)
   l = length(msg);
   mat = repmat(' ', depth, l);   % zigzag matrix
   row = 1;
   down = true;
   for col = 1:l
      mat(row, col) = msg(col);

      if down
         row = row + 1;
         if row > depth
            row = depth - 1;
            down = false;
         end
      else
```

```
row = row - 1;
          if row < 1
             row = 2;
             down = true;
          end
       end
   end

   enc = reshape(mat', 1, []);     % read row-wise
   enc(enc == ' ') = [];           % remove spaces
end

% ---------------- Decryption ----------------
function dec = decode(enc, depth)
   l = length(enc);
   mat = repmat(' ', depth, l);

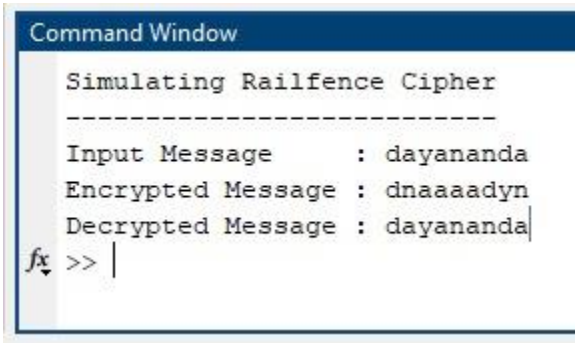
   % Mark zigzag positions
   row = 1; down = true;
   for col = 1:l
      mat(row, col) = '*';
      if down
         row = row + 1;
         if row > depth
            row = depth - 1;
            down = false;
         end
      else
         row = row - 1;
         if row < 1
            row = 2;
            down = true;
         end
      end
   end

   % Fill the marked positions with ciphertext
   idx = 1;
   for i = 1:depth
      for j = 1:l
         if mat(i, j) == '*'
            mat(i, j) = enc(idx);
            idx = idx + 1;
         end
      end
   end

   % Read zigzag again to get plaintext
   dec = '';
   row = 1; down = true;
```

```
    for col = 1:l
        dec = [dec mat(row, col)];

  if down
          row = row + 1;
          if row > depth
             row = depth - 1;
             down = false;
          end
        else
          row = row - 1;
          if row < 1
             row = 2;
             down = true;
          end
        end
    end
end
```



2B: ROW AND COLUMN TRANSFORMATION TECHNIQUE

ALGORITHM:

1. Consider the plain text hello world, and let us apply the simple columnar transposition technique as shown below

h	e	l	l
o	w	o	r
l	d		

2. The plain text characters are placed horizontally and the cipher text is created with vertical format as: holewdlolr.

3. Now, the receiver has to use the same table to decrypt the cipher text to plain text.

```
function TransCipher()
    % Transposition Cipher in MATLAB (Encryption + Decryption)

    % Input
    pl = input('Enter the plain text: ', 's');

    % Step 1: Remove spaces
    s = strrep(pl, ' ', '');  % direct space removal
    fprintf('Text without spaces: %s\n', s);

    % Step 2: Initialize encryption matrix
    k = length(s);
    col = 4;                                % fixed column size
    row = ceil(k / col);                    % ensure enough rows
    ch = repmat('#', row, col);             % fill with '#'

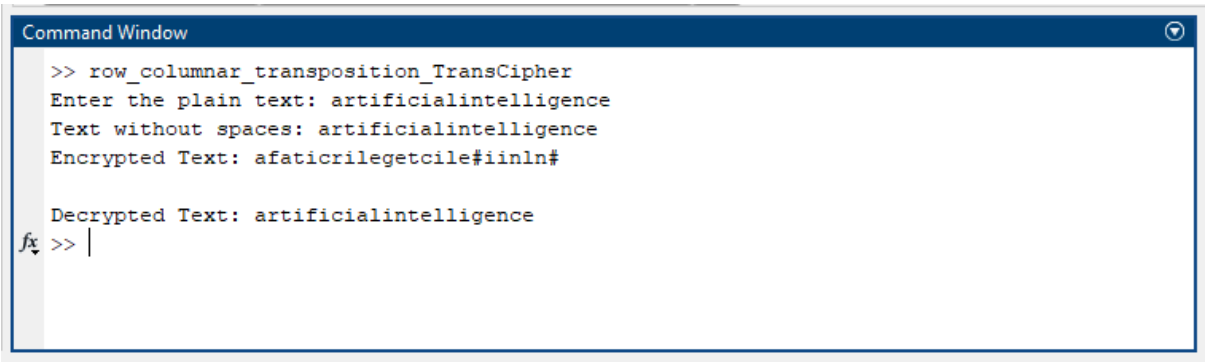
    % Fill matrix row-wise
    l = 1;
    for i = 1:row
        for j = 1:col
            if l <= k
                ch(i,j) = s(l);
                l = l + 1;
            end
        end
    end
    % Step 3: Transpose (columnar arrangement)
    trans = ch.';   % transpose in MATLAB

    % Step 4: Read column-wise (encryption)
    enc = '';
    for i = 1:col
        for j = 1:row
            enc = [enc trans(i,j)];
        end
    end
    fprintf('Encrypted Text: %s\n', enc);

    % ---------------- Decryption ----------------
    % Step 5: Fill matrix column-wise from ciphertext
    decMat = repmat('#', col, row);
    l = 1;
    for i = 1:col
        for j = 1:row
            decMat(i,j) = enc(l);
            l = l + 1;
        end
    end
    % Step 6: Transpose back
    decMat2 = decMat.';

    % Step 7: Read row-wise to reconstruct message
    dec = '';
    for i = 1:row
        for j = 1:col
            dec = [dec decMat2(i,j)];
        end
    end
```

```
    % Remove padding '#'
    dec = strrep(dec, '#', '');
fprintf('\nDecrypted Text: %s\n', dec);
end
```



Assignment:

1. Run the program for at least three different messages with varying rail values (e.g., 2, 3, 4).

2. Compare the cipher texts generated and explain how changing the key (number of rails) affects encryption.

Signature:

Date:

Program-3: To implement a Simple Data Encryption Standard algorithm

Algorithm: DES Encryption & Decryption (MATLAB)

1. Input Stage

1.1 Prompt the user to enter plaintext (8 ASCII characters).
1.2 Prompt the user to enter key (8 ASCII characters).
1.3 Validate that both are exactly 8 characters (64 bits).

2. Convert Inputs to Binary

2.1 Convert plaintext characters to an 8×8 binary matrix (each char → 8 bits).
2.2 Reshape into a 1×64 binary vector.
2.3 Convert the key string into the same 1×64 binary vector format.

3. Key Scheduling (Generate 16 Subkeys)

3.1 Apply Permuted Choice 1 (PC-1) to key → get 56-bit key.
3.2 Split into C (left 28 bits) and D (right 28 bits).
3.3 For each round i = 1 to 16:

- Left circular shift C and D by an amount given in the shift schedule.
- Combine C and D → 56 bits.
- Apply Permuted Choice 2 (PC-2) to get a 48-bit subkey Ki.
  3.4 Store all 16 subkeys in order.

4. Encryption Process

4.1 Apply Initial Permutation (IP) to plaintext bits → split into L0 (first 32 bits) and R0 (last 32 bits).
4.2 For each round i = 1 to 16:

- Expansion (E): Expand 32-bit Ri-1 to 48 bits using the expansion table.
- XOR with subkey Ki.
- Substitution (S-boxes): Split into eight 6-bit blocks, pass each through its corresponding S-box, producing 32 bits.
- Permutation (P): Rearrange 32 bits according to permutation table P.
- Feistel step:
- Li = Ri-1
- Ri = Li-1 XOR P_output

4.3 After round 16, swap the left and right halves.
4.4 Apply Inverse Initial Permutation (IP^{-1}) to produce the 64-bit ciphertext.

5. Ciphertext Output

5.1 Group bits into bytes, convert to decimal, then to hexadecimal string.
5.2 Display ciphertext in hex.

6. Decryption Process

6.1 Same as encryption, but use subkeys in reverse order (K16, K15, …, K1).
6.2 Apply IP → run 16 rounds → swap halves → apply IP^{-1}.
6.3 Convert resulting bits into ASCII characters.
6.4 Display recovered plaintext.

```
% DES Encryption & Decryption in MATLAB
% ------------------------------------------------------
clear; clc;
% --- User Input ---
plaintext_str = input('Enter plaintext (8 ASCII characters): ', 's');
key_str       = input('Enter key (8 ASCII characters): ', 's');

if length(plaintext_str) ~= 8 || length(key_str) ~= 8
    error('Both plaintext and key must be exactly 8 ASCII characters.');
end

% Convert to binary vectors
plaintext_bits = reshape(de2bi(uint8(plaintext_str), 8, 'left-msb')', 1, []);
key_bits       = reshape(de2bi(uint8(key_str), 8, 'left-msb')', 1, []);

% --- DES Tables ---
IP = [58 50 42 34 26 18 10 2 60 52 44 36 28 20 12 4 ...
      62 54 46 38 30 22 14 6 64 56 48 40 32 24 16 8 ...
      57 49 41 33 25 17  9 1 59 51 43 35 27 19 11 3 ...
      61 53 45 37 29 21 13 5 63 55 47 39 31 23 15 7];

IP_inv = [40 8 48 16 56 24 64 32 39 7 47 15 55 23 63 31 ...
          38 6 46 14 54 22 62 30 37 5 45 13 53 21 61 29 ...
          36 4 44 12 52 20 60 28 35 3 43 11 51 19 59 27 ...
          34 2 42 10 50 18 58 26 33 1 41  9 49 17 57 25];

E = [32 1 2 3 4 5 4 5 6 7 8 9 8 9 10 11 12 13 ...
     12 13 14 15 16 17 16 17 18 19 20 21 ...
     20 21 22 23 24 25 24 25 26 27 28 29 ...
     28 29 30 31 32 1];

P = [16 7 20 21 29 12 28 17 1 15 23 26 5 18 31 10 ...
     2 8 24 14 32 27 3 9 19 13 30 6 22 11 4 25];

PC1 = [57 49 41 33 25 17 9 1 58 50 42 34 26 18 ...
       10 2 59 51 43 35 27 19 11 3 60 52 44 36 ...
       63 55 47 39 31 23 15 7 62 54 46 38 30 22 ...
       14 6 61 53 45 37 29 21 13 5 28 20 12 4];

PC2 = [14 17 11 24 1 5 3 28 15 6 21 10 23 19 12 4 ...
       26 8 16 7 27 20 13 2 41 52 31 37 47 55 30 40 ...
       51 45 33 48 44 49 39 56 34 53 46 42 50 36 29 32];
shifts = [1 1 2 2 2 2 2 2 1 2 2 2 2 2 2 1];

% --- S-boxes ---
S_boxes = { ...
```

```
[14 4 13 1 2 15 11 8 3 10 6 12 5 9 0 7;
 0 15 7 4 14 2 13 1 10 6 12 11 9 5 3 8;
 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0;
 15 12 8 2 4 9 1 7 5 11 3 14 10 0 6 13],
[15 1 8 14 6 11 3 4 9 7 2 13 12 0 5 10;
 3 13 4 7 15 2 8 14 12 0 1 10 6 9 11 5;
 0 14 7 11 10 4 13 1 5 8 12 6 9 3 2 15;
 13 8 10 1 3 15 4 2 11 6 7 12 0 5 14 9],

[10 0 9 14 6 3 15 5 1 13 12 7 11 4 2 8;
 13 7 0 9 3 4 6 10 2 8 5 14 12 11 15 1;
 13 6 4 9 8 15 3 0 11 1 2 12 5 10 14 7;
 1 10 13 0 6 9 8 7 4 15 14 3 11 5 2 12],
[7 13 14 3 0 6 9 10 1 2 8 5 11 12 4 15;
 13 8 11 5 6 15 0 3 4 7 2 12 1 10 14 9;
 10 6 9 0 12 11 7 13 15 1 3 14 5 2 8 4;
 3 15 0 6 10 1 13 8 9 4 5 11 12 7 2 14],
[2 12 4 1 7 10 11 6 8 5 3 15 13 0 14 9;
 14 11 2 12 4 7 13 1 5 0 15 10 3 9 8 6;
 4 2 1 11 10 13 7 8 15 9 12 5 6 3 0 14;
 11 8 12 7 1 14 2 13 6 15 0 9 10 4 5 3],
[12 1 10 15 9 2 6 8 0 13 3 4 14 7 5 11;
 10 15 4 2 7 12 9 5 6 1 13 14 0 11 3 8;
 9 14 15 5 2 8 12 3 7 0 4 10 1 13 11 6;
 4 3 2 12 9 5 15 10 11 14 1 7 6 0 8 13],
[4 11 2 14 15 0 8 13 3 12 9 7 5 10 6 1;
 13 0 11 7 4 9 1 10 14 3 5 12 2 15 8 6;
 1 4 11 13 12 3 7 14 10 15 6 8 0 5 9 2;
 6 11 13 8 1 4 10 7 9 5 0 15 14 2 3 12],
[13 2 8 4 6 15 11 1 10 9 3 14 5 0 12 7;
 1 15 13 8 10 3 7 4 12 5 6 11 0 14 9 2;
 7 11 4 1 9 12 14 2 0 6 10 13 15 3 5 8;
 2 1 14 7 4 10 8 13 15 12 9 0 3 5 6 11]};

% --- Key Schedule ---
key56 = key_bits(PC1);
C = key56(1:28); D = key56(29:56);
subkeys = zeros(16,48);

for i = 1:16
    C = circshift(C, [0 -shifts(i)]);
    D = circshift(D, [0 -shifts(i)]);
    CD = [C D];                 % FIX: store concat in a variable
    subkeys(i,:) = CD(PC2);     % then index it
end
% --- Encryption ---
cipher_bits = DES_rounds(plaintext_bits, subkeys, IP, IP_inv, E, P, S_boxes);
cipher_hex = reshape(dec2hex(bi2de(reshape(cipher_bits, 8, [])', 'left-msb'))',1,[]);
fprintf('Ciphertext (Hex): %s\n', cipher_hex);
```

```
% --- Decryption ---
plain_bits_recovered = DES_rounds(cipher_bits, subkeys(end:-1:1,:), IP, IP_inv, E, P,
S_boxes);
plain_ascii = char(bi2de(reshape(plain_bits_recovered, 8, [])', 'left-msb'))';
fprintf('Recovered Plaintext: %s\n', plain_ascii);

% --- Function ---
function output_bits = DES_rounds(input_bits, subkeys, IP, IP_inv, E, P, S_boxes)
    IP_out = input_bits(IP);
    L = IP_out(1:32);
    R = IP_out(33:64);
    for round = 1:16
        ER = R(E);
        x = xor(ER, subkeys(round,:));
        s_out = [];
        for box = 1:8
            six_bits = x(6*(box-1)+1 : 6*box);
            row = bi2de([six_bits(1) six_bits(6)], 'left-msb') + 1;
            col = bi2de(six_bits(2:5), 'left-msb') + 1;
            val = S_boxes{box}(row, col);
            s_out = [s_out de2bi(val, 4, 'left-msb')];
        end
        p_out = s_out(P);
        newR = xor(L, p_out);
        L = R;
        R = newR;
    end
    pre_output = [R L];
    output_bits = pre_output(IP_inv);
end
```

```
Command Window
  Enter plaintext (8 ASCII characters): helloall
  Enter key (8 ASCII characters): mysecret
  Ciphertext (Hex): 29A89B048FE93AA7
  Recovered Plaintext: helloall
fx >>
```

Assignment: 1.Develop different S-boxes and comment on its effect on encryption and decryption process.

2. Develop different IP and IP^{-1} and comment on its effect on encryption and decryption process.

Signature:

Date:

Program-4: To implement a simplified Advanced Encryption Standard algorithm

Algorithm for S-AES Encryption and Decryption

Input:

- 16-bit plaintext (in hex).
- 16-bit key (in hex).

Output:

- 16-bit ciphertext (in hex).
- Decrypted plaintext (in hex).

Step 1: Input

1. Read 16-bit plaintext from user.
2. Read 16-bit key from user.
3. Convert both plaintext and key from hex to binary form.

Step 2: Key Expansion

1. Split the 16-bit key into two 8-bit words w0 and w1.
2. Compute additional words using key expansion:
    - $w2 = w0 \oplus g(w1)$
    - $w3 = w2 \oplus w1$
    - $w4 = w2 \oplus g(w3)$
    - $w5 = w4 \oplus w3$
      where function g = RotateNibble + SubNibble + XOR with Round Constant (RCON).
3. Round keys are formed as:
    - K0 = w0 || w1
    - K1 = w2 || w3
    - K2 = w4 || w5

Step 3: Encryption

1. **Round 0 (Initial AddRoundKey)**:
    - State = Plaintext $\oplus$ K0
2. **Round 1**:
    - Apply **Nibble Substitution** (using S-Box).
    - Apply **ShiftRow** (circular left shift of 2nd row).
    - Apply **MixColumn** (matrix multiplication in GF(2^4)).

- AddRoundKey with K1.

3. **Round 2**:
   - Apply **Nibble Substitution**.
   - Apply **ShiftRow**.
   - AddRoundKey with K2.
4. Output of Round 2 is the **Ciphertext**.

Step 4: Decryption

1. **Inverse Round 2**:
   - AddRoundKey with K2.
   - InverseShiftRow.
   - InverseNibbleSubstitution.
2. **Inverse Round 1**:
   - AddRoundKey with K1.
   - InverseMixColumn.
   - InverseShiftRow.
   - InverseNibbleSubstitution.
3. **Inverse Round 0**:
   - AddRoundKey with K0.
4. Output is the **Recovered Plaintext**.

Step 5: Display Results

1. Print Ciphertext (in hex).
2. Print Decrypted Plaintext (in hex).

MATLAB CODE:

```
% Simplified AES (S-AES) – Lab program (16-bit block, 16-bit key)
% Prompts user for 4-hex plaintext and key; prints ciphertext and decrypted
text.
% Spec aligned with your reference: S-Box/Inv S-Box, MixColumn [[1 4;4 1]], Inv
[[9 2;2 9]],
% key expansion with RotNib/SubNib and RCON1=0x80, RCON2=0x30.  (See Appendix D)

clc;
fprintf('--- S-AES (16-bit) ---\n');
pt_hex  = upper(strtrim(input('Enter 16-bit plaintext (4 hex chars, e.g., ABCD):
','s')));
key_hex = upper(strtrim(input('Enter 16-bit key       (4 hex chars, e.g., 3F27):
','s')));
assert(all(isstrprop(pt_hex,'xdigit')) && numel(pt_hex)<=4, 'Plaintext must be
hex (≤4 chars).');
assert(all(isstrprop(key_hex,'xdigit')) && numel(key_hex)<=4, 'Key must be hex
(≤4 chars).');

pt  = hex4_to_nibbles(pt_hex);   % [S00 S10 S01 S11] = [n0 n1 n2 n3]
key = hex4_to_nibbles(key_hex);

[K0,K1,K2] = key_expand(key);
```

```
ct  = s_encrypt(pt,  K0,K1,K2);
dec = s_decrypt(ct,  K0,K1,K2);

fprintf('Ciphertext          : %s\n', nibbles_to_hex4(ct));
fprintf('Decrypted Plaintext : %s\n', nibbles_to_hex4(dec));


%% ---------------- Core S-AES ----------------
function C = s_encrypt(S, K0,K1,K2)
    S = add_key(S,K0);          % Round 0
    S = sub_nibbles(S);
    S = shift_row(S);
    S = mix_columns(S);
    S = add_key(S,K1);          % Round 1
    S = sub_nibbles(S);
    S = shift_row(S);
    S = add_key(S,K2);          % Round 2 (no MixColumn)
    C = S;
end
function P = s_decrypt(C, K0,K1,K2)
    S = add_key(C,K2);          % Inv Round 2
    S = shift_row(S);           % ISR == SR for 2 columns
    S = inv_sub_nibbles(S);

    S = add_key(S,K1);          % Inv Round 1
    S = inv_mix_columns(S);
    S = shift_row(S);
    S = inv_sub_nibbles(S);
    S = add_key(S,K0);          % Inv Round 0
    P = S;
end

%% --------------- Round functions (nibble-level) ---------------
function out = add_key(a,b)
    out = bitxor(double(a), double(b));
end


function out = sub_nibbles(a)
    sbox = [9 4 10 11 13 1 8 5 6 2 0 3 12 14 15 7]; % Table D.1a
    out  = arrayfun(@(x) sbox(x+1), a);
end

function out = inv_sub_nibbles(a)
    invs = [10 5 9 11 1 7 8 15 6 0 2 3 12 4 13 14]; % Table D.1b
    out  = arrayfun(@(x) invs(x+1), a);
end

function out = shift_row(a)
    % a = [S00 S10 S01 S11]  (column-major nibbles)
    % Shift 2nd row left by 1: [S10 S11] -> [S11 S10]  => swap a(2) <-> a(4)
    out = [a(1) a(4) a(3) a(2)];
end

function out = mix_columns(a)
    % Per-column: [S'00;S'10] = [1 4; 4 1]*[S00;S10]   and same for right column
    out = zeros(1,4);
    out(1) = bitxor(a(1), gf_mul(4,a(2)));
    out(2) = bitxor(gf_mul(4,a(1)), a(2));
```

```
    out(3) = bitxor(a(3), gf_mul(4,a(4)));
    out(4) = bitxor(gf_mul(4,a(3)), a(4));
end

function out = inv_mix_columns(a)
    % [S'00;S'10] = [9 2; 2 9]*[S00;S10]
    out = zeros(1,4);
    out(1) = bitxor(gf_mul(9,a(1)), gf_mul(2,a(2)));
    out(2) = bitxor(gf_mul(2,a(1)), gf_mul(9,a(2)));
    out(3) = bitxor(gf_mul(9,a(3)), gf_mul(2,a(4)));
    out(4) = bitxor(gf_mul(2,a(3)), gf_mul(9,a(4)));
end

%% ------------------- Key expansion (Appendix D.3) -------------------
function [K0,K1,K2] = key_expand(k)
    % k = [k0 k1 k2 k3] nibbles; bytes are w0=[k0 k1], w1=[k2 k3]
    w0 = uint8(bitshift(k(1),4) + k(2));
    w1 = uint8(bitshift(k(3),4) + k(4));

    g1 = bitxor(uint8(128), subNib_byte(rotNib(w1)));   % RCON1=0x80
    w2 = bitxor(w0, g1);
    w3 = bitxor(w2, w1);

    g2 = bitxor(uint8(48),  subNib_byte(rotNib(w3)));   % RCON2=0x30
    w4 = bitxor(w2, g2);
    w5 = bitxor(w4, w3);

    K0 = [bitshift(w0,-4), bitand(w0,15), bitshift(w1,-4), bitand(w1,15)];
    K1 = [bitshift(w2,-4), bitand(w2,15), bitshift(w3,-4), bitand(w3,15)];
    K2 = [bitshift(w4,-4), bitand(w4,15), bitshift(w5,-4), bitand(w5,15)];
End


function y = rotNib(b)          % rotate two nibbles in a byte
    y = bitor(bitshift(b,4), bitshift(b,-4));  % implicit & 0xFF in uint8
end

function y = subNib_byte(b)
    sbox = uint8([9 4 10 11 13 1 8 5 6 2 0 3 12 14 15 7]);
    hi = bitshift(b,-4); lo = bitand(b,15);
    y  = uint8(bitshift(sbox(double(hi)+1),4) + sbox(double(lo)+1));
end

%% ---------------- GF(2^4) multiply via table (Appendix D.2) ----------------
function z = gf_mul(m,a)
    % m,a in 0..15 (nibbles)
    % Full table from GF(2^4) mod x^4+x+1  (rows: multiplier, cols:
multiplicand)
    persistent T;
    if isempty(T)
        T = [ ...
        0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;   % 0*
        0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; % 1*
        0 2 4 6 8 10 12 14 3 1 7 5 11 9 15 13; % 2*
        0 3 6 5 12 15 10 9 11 8 13 14 7 4 1 2;  % 3*
        0 4 8 12 3 7 11 15 6 2 14 10 5 1 13 9;  % 4*
        0 5 10 15 7 2 13 8 14 11 4 1 9 12 3 6;  % 5*
        0 6 12 10 11 13 7 1 5 3 9 15 14 8 2 4;  % 6*
        0 7 14 9 15 8 1 6 13 10 3 4 2 5 12 11;  % 7*
        0 8 3 11 6 14 5 13 12 4 15 7 10 2 9 1;  % 8*
```

```
        0 9 1 8 2 11 3 10 4 13 5 12 6 15 7 14;  % 9*
        0 10 7 13 14 4 9 3 15 5 8 2 1 11 6 12;  % A*
        0 11 5 14 10 1 15 4 7 12 2 9 13 6 8 3;  % B*
        0 12 11 7 5 9 14 2 10 6 1 13 15 3 4 8;  % C*
        0 13 9 4 1 12 8 5 2 15 11 6 3 14 10 7;  % D*
        0 14 15 1 13 3 2 12 9 7 6 8 4 10 11 5;  % E*
        0 15 13 2 9 6 4 11 1 14 12 3 8 7 5 10]; % F*
    end
    z = T(m+1, a+1);
end

%% ----------------- I/O helpers -----------------
function n = hex4_to_nibbles(h)
    if numel(h) < 4, h = pad(h,4,'left','0'); end
    v = hex2dec(h);
    n = [bitand(bitshift(v,-12),15), bitand(bitshift(v,-8),15), ...
         bitand(bitshift(v,-4),15),  bitand(v,15)];
end

function h = nibbles_to_hex4(n)
    v = bitshift(n(1),12) + bitshift(n(2),8) + bitshift(n(3),4) + n(4);
    h = upper(sprintf('%04X', v));
end
```

```
Command Window
--- S-AES (16-bit) ---
Enter 16-bit plaintext (4 hex chars, e.g., ABCD): 12cd
Enter 16-bit key       (4 hex chars, e.g., 3F27): e34f
Ciphertext          : 00AD
Decrypted Plaintext : 12CD
fx >>
```

Assignment: Write a MATLAB program to simulate a **password-based login system** using AES encryption.

- The system stores a **16-bit encrypted password**.
- When a user enters their password, the program encrypts it and compares it with the stored ciphertext.
- If they match → access granted, else → access denied.

Real-time link: Used in banking ATMs, smartphones, or any secure login.

Signature:

Date:

Program 5: Secure Online Transactions (like e-commerce or banking) using the RSA algorithm

Algorithm: Secure Online Transactions using RSA

Step 1: Key Generation (Bank Side)

1. Choose two large prime numbers `p` and `q`.
2. Compute modulus:

$$n = p \times q$$

3. Compute Euler's totient:

$$\phi = (p-1)(q-1)$$

4. Select public exponent `e` such that:

$$1 < e < \phi \quad \text{and} \quad \gcd(e, \phi) = 1$$

5. Compute private exponent `d` as the modular inverse of `e` (mod φ):

$$d \times e \equiv 1 \pmod{\phi}$$

6. Publish the **public key** `(e, n)` and keep the **private key** `(d, n)` secret.

Step 2: Transaction Preparation (User Side)

1. Collect sensitive transaction data:
   - Example: `Card Number | Amount`.
2. Convert the transaction message into its **ASCII numeric representation**.
3. Encrypt each number using RSA encryption with the **bank's public key**:

$$C = M^e \mod n$$

   where `M` is the message block, `C` is ciphertext.
4. Send the encrypted ciphertext to the bank server.

Step 3: Transaction Decryption (Bank Side)

1. Receive ciphertext from the customer.
2. Decrypt each number using RSA decryption with the **bank's private key**:

$$M = C^d \mod n$$

3. Convert the numeric values back to ASCII characters.
4. Reconstruct the original message (e.g., `1234567890123456 | Rs5000`).

Step 4: Verification and Processing

1. Bank verifies the transaction details (valid card number, sufficient balance, etc.).
2. If valid → approve transaction; else → reject.

MATLAB CODE

```
%Secure Online Transactions (like e-commerce or banking) using the RSA
algorithm.
%
% It shows how sensitive data (e.g., card number, amount) can be securely
transmitted from a customer to a bank server.


clc;
clear;

%% Step 1: Bank Generates RSA Keys
p = 47;
q = 59;
n = p * q;
phi = (p - 1) * (q - 1);

% Public exponent
e = 3;
while gcd(e, phi) ~= 1
    e = e + 2;
end

% Private exponent
d = modInverse(e, phi);

fprintf('Bank Public Key (e,n): (%d, %d)\n', e, n);
fprintf('Bank Private Key (d,n): (%d, %d)\n\n', d, n);

%% Step 2: User Prepares Transaction
cardNumber = '1234567890123456';  % Sample 16-digit card number
amount = 'Rs5000';

message = [cardNumber '|' amount];  % Combine data
fprintf('Original Transaction: %s\n', message);

% Convert message to ASCII
asciiData = double(message);
```

```
% Encrypt with bank's public key
cipher = arrayfun(@(x) modexp(x, e, n), asciiData);
fprintf('Encrypted Data Sent to Bank:\n');
disp(cipher);

%% Step 3: Bank Decrypts Transaction
decrypted = arrayfun(@(x) modexp(x, d, n), cipher);
decryptedMessage = char(decrypted);

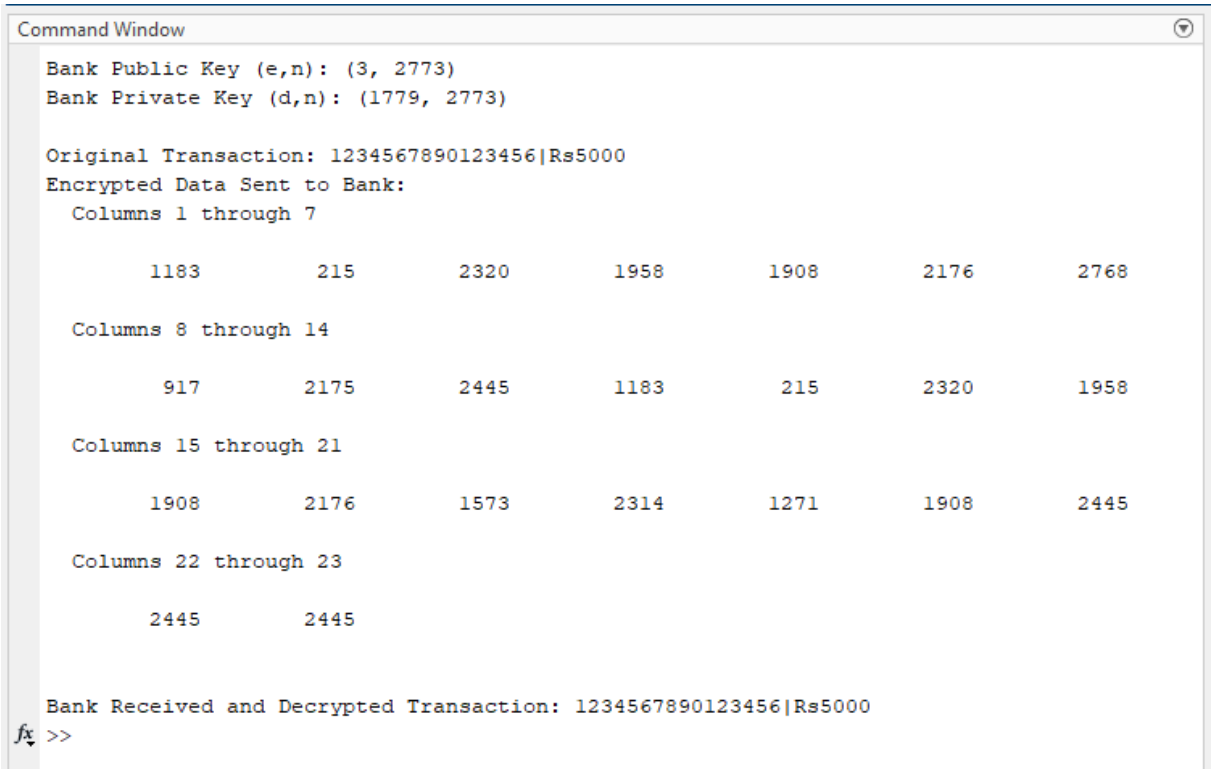
fprintf('\nBank Received and Decrypted Transaction: %s\n', decryptedMessage);

%% Helper Functions
function g = gcd(a, b)
    while b ~= 0
        temp = b;
        b = mod(a, b);
        a = temp;
    end
    g = a;
end

function d = modInverse(e, phi)
    [g, x, ~] = extendedGCD(e, phi);
    if g ~= 1
        error('Modular inverse does not exist.');
    else
        d = mod(x, phi);
    end
end

function [g, x, y] = extendedGCD(a, b)
    if b == 0
        g = a; x = 1; y = 0;
    else
        [g, x1, y1] = extendedGCD(b, mod(a, b));
        x = y1;
        y = x1 - floor(a/b) * y1;
    end
end

function result = modexp(base, exponent, modval)
    result = 1;
    base = mod(base, modval);
    while exponent > 0
        if mod(exponent, 2) == 1
            result = mod(result * base, modval);
        end
        exponent = floor(exponent / 2);
        base = mod(base^2, modval);
    end
end
```



Assignment: Generate RSA keys to establish a secure session key between the user's browser and the website, for secure email transactions

Signature:

Date:

Program 6: Implement Diffie-Hellman Key Exchange to establish session keys in a mobile application

Algorithm for Diffie–Hellman Key Exchange

1. Step 1: Input a prime number p.
2. Step 2: Input a primitive root g of p.
3. Step 3: Sender (Alice) chooses a private key x, where x < p.
4. Step 4: Compute Alice's public value: R1 = (g^x) mod p.
5. Step 5: Receiver (Bob) chooses a private key y, where y < p.
6. Step 6: Compute Bob's public value: R2 = (g^y) mod p.
7. Step 7: Exchange R1 and R2 between Alice and Bob.
8. Step 8: Both compute the shared secret key:
   - At sender's side: K1 = (R2^x) mod p
   - At receiver's side: K2 = (R1^y) mod p
9. Step 9: Since K1 = K2, the shared secret key is established.

Manual Calculation Example of Diffie–Hellman Key Exchange

Let us consider an example with small numbers to demonstrate the Diffie–Hellman key exchange step by step.

Public Parameters
Prime number (p) = 23
Primitive root (g) = 5
Private Keys
Alice (Sender) chooses x = 6 (private key).
Bob (Receiver) chooses y = 15 (private key).

Step 1: Compute Alice's Public Value (R1)
R1 = g^x mod p = 5^6 mod 23
5^2 = 25 ≡ 2 (mod 23)
5^4 = (5^2)^2 = 2^2 = 4 (mod 23)
5^6 = 5^4 × 5^2 = 4 × 2 = 8 (mod 23)
Thus, R1 = 8.

Step 2: Compute Bob's Public Value (R2)
R2 = g^y mod p = 5^15 mod 23
Using binary method: 15 = 8 + 4 + 2 + 1
5^1 = 5, 5^2 = 2, 5^4 = 4, 5^8 = 16 (mod 23)
Now, 5^15 = 5^8 × 5^4 × 5^2 × 5^1 ≡ 16 × 4 × 2 × 5 (mod 23)
= (16×4=18) ×2=13 ×5=19 (mod 23)
Thus, R2 = 19.

Step 3: Alice Computes Shared Key (K1)
K1 = R2^x mod p = 19^6 mod 23
19 ≡ -4 (mod 23). So, compute (-4)^6.
19^2 = 16, 19^4 = 3, so 19^6 = 19^4 × 19^2 = 3 × 16 = 48 ≡ 2 (mod 23).

Thus, K1 = 2.

Step 4: Bob Computes Shared Key (K2)
K2 = R1^y mod p = 8^15 mod 23
8^2 = 18, 8^4 = 2, 8^8 = 4 (mod 23)
So, 8^15 = 8^8 × 8^4 × 8^2 × 8^1 = 4 × 2 × 18 × 8 (mod 23)
= (4×2=8) ×18=6 ×8=48 ≡ 2 (mod 23).
Thus, K2 = 2.

Conclusion
Both Alice and Bob computed the same shared secret key:
K1 = K2 = 2
Therefore, the Diffie–Hellman key exchange was successful.

MATLAB CODE

```
clc;
clear;

% Step 1: Input prime number p
p = input('Enter prime number: ');

% Step 2: Input primitive root g
g = input(['Enter primitive root of ', num2str(p), ': ']);

% Step 3: Input private key x (Sender side)
x = input(['Enter value for x less than ', num2str(p), ': ']);

% Compute R1 = g^x mod p
R1 = powermod(g, x, p);
fprintf('R1 = %d\n', R1);

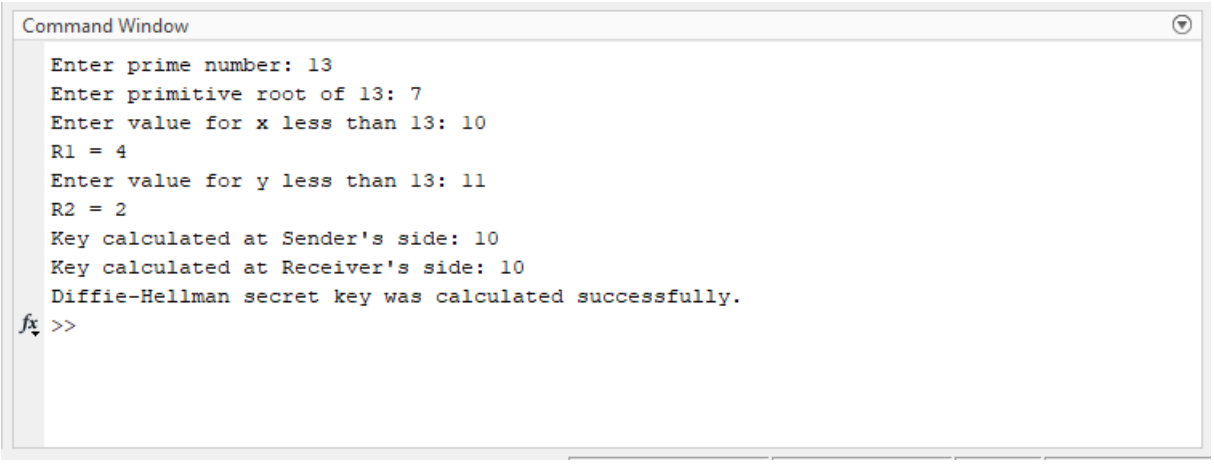
% Step 4: Input private key y (Receiver side)
y = input(['Enter value for y less than ', num2str(p), ': ']);

% Compute R2 = g^y mod p
R2 = powermod(g, y, p);
fprintf('R2 = %d\n', R2);

% Step 5: Compute secret key at sender's side
K1 = powermod(R2, x, p);
fprintf('Key calculated at Sender''s side: %d\n', K1);

% Step 6: Compute secret key at receiver's side
K2 = powermod(R1, y, p);
fprintf('Key calculated at Receiver''s side: %d\n', K2);

disp('Diffie-Hellman secret key was calculated successfully.');
```



Assignment:

1. Modify the program so that:Each side generates a **random private key** automatically. The program only asks for p and g.
2. Extend the program to simulate **three users** (Alice, Bob, and Charlie) agreeing on a common secret key using Diffie–Hellman.
3. Use the generated secret key to perform a **simple encryption/decryption** (e.g., Caesar cipher, XOR cipher) of a message.

Signature:

Date:

Program-7: Calculate the message digest of a text using the SHA-1 algorithm.

Algorithm

1. **Input**: A message string M.
2. Convert the input message into binary (ASCII → 8-bit binary).
3. Append a single **1** bit to the message.
4. Append **0** bits until the total length ≡ 448 (mod 512).
5. Append the **64-bit representation** of the original message length (big-endian).
6. Initialize five hash values (H0–H4) with fixed constants:
   - H0 = 67452301
   - H1 = EFCDAB89
   - H2 = 98BADCFE
   - H3 = 10325476
   - H4 = C3D2E1F0
7. Break the padded message into **512-bit blocks**.
8. For each block:
   a. Break into 16 words W[0..15] of 32 bits each.
   b. Extend W[16..79] using XOR and rotation.
   c. Initialize variables a, b, c, d, e = H0..H4.
   d. Perform 80 rounds with round functions and constants.
   e. Update H0..H4 with the results.
9. Concatenate H0, H1, H2, H3, H4 (in hex) → final **SHA-1 digest**.

Manual Example

Message: `"abc"`

1. **ASCII → Binary**
   - 'a' = 97 = `01100001`
   - 'b' = 98 = `01100010`
   - 'c' = 99 = `01100011`
     → Binary stream:
     `01100001 01100010 01100011`
2. Append **1 bit** and pad with **0's** until total length ≡ 448 (mod 512).
3. Append **64-bit length** (24 in binary).
4. First **512-bit block** is formed.
5. Break into **16 words of 32 bits**:
   - Example: W[0] = `01100001011000100110001100000000`
6. Initialize hash values H0..H4.
7. Process block with **80 rounds**.
8. Update hash values.

Final Digest

SHA1("abc") =
a9993e364706816aba3e25717850c26c9cd0d89d

MATLAB CODE

```
clc;
clear;

% Input string
msg = input('Enter the string: ', 's');

% --- Step 1: Convert to binary and pad ---
msg_bin = reshape(dec2bin(msg,8).'-'0',1,[]); % ASCII → binary
mlen = length(msg_bin);

% Append 1 and pad with zeros until length ≡ 448 mod 512
msg_bin = [msg_bin 1];
while mod(length(msg_bin),512) ~= 448
    msg_bin = [msg_bin 0];
end

% Append 64-bit length (big-endian)
len64 = dec2bin(mlen,64)-'0';
msg_bin = [msg_bin len64];

% --- Step 2: Initialize hash values (as uint32) ---
h = uint32([hex2dec('67452301'), hex2dec('EFCDAB89'), ...
            hex2dec('98BADCFE'), hex2dec('10325476'), ...
            hex2dec('C3D2E1F0')]);

% --- Step 3: Process each 512-bit chunk ---
for chunk = 1:512:length(msg_bin)
    block = msg_bin(chunk:chunk+511);

    % Break into 16 words of 32 bits
    w = zeros(1,80,'uint32');
    for i=0:15
        word = block(i*32+1:(i+1)*32);
        w(i+1) = uint32(bin2dec(char(word+'0')));
    end

    % Extend to 80 words
    for i=17:80
        w(i) = bitrotate(bitxor(bitxor(bitxor(w(i-3),w(i-8)),w(i-14)),w(i-
16)),1);
    end

    % Working variables
    a=h(1); b=h(2); c=h(3); d=h(4); e=h(5);

    for i=1:80
        if i<=20
            f = bitor(bitand(b,c), bitand(bitcmp(b),d));
            k = uint32(hex2dec('5A827999'));
        elseif i<=40
            f = bitxor(bitxor(b,c),d);
            k = uint32(hex2dec('6ED9EBA1'));
```

```
        elseif i<=60
            f = bitor(bitor(bitand(b,c), bitand(b,d)), bitand(c,d));
            k = uint32(hex2dec('8F1BBCDC'));
        else
            f = bitxor(bitxor(b,c),d);
            k = uint32(hex2dec('CA62C1D6'));
        end

        temp = mod(uint64(bitrotate(a,5)) + uint64(f) + uint64(e) + uint64(k) +
uint64(w(i)), 2^32);
        temp = uint32(temp);

        e=d; d=c; c=bitrotate(b,30); b=a; a=temp;
    end

    % Update hash
    h(1) = uint32(mod(uint64(h(1)) + uint64(a),2^32));
    h(2) = uint32(mod(uint64(h(2)) + uint64(b),2^32));
    h(3) = uint32(mod(uint64(h(3)) + uint64(c),2^32));
    h(4) = uint32(mod(uint64(h(4)) + uint64(d),2^32));
    h(5) = uint32(mod(uint64(h(5)) + uint64(e),2^32));
end

% --- Step 4: Output digest ---
digest = lower([dec2hex(h(1),8) dec2hex(h(2),8) dec2hex(h(3),8) dec2hex(h(4),8)
dec2hex(h(5),8)]);
fprintf('SHA-1 Digest = %s\n', digest);

% --- Helper function: rotate left ---
function r = bitrotate(x,n)
    r = bitor(bitshift(x,n,'uint32'), bitshift(x,int32(n)-32,'uint32'));
end
```

```
Command Window
  Enter the string: dicipline
  SHA-1 Digest = 365b61eb8fc767af4b3df39371c78c3f72b7095d
fx >>
```

Assignment: 1. Avalanche Effect Experiment

- Hash a message (e.g., `"hello"`).
- Flip one bit and hash again.
- Count how many bits differ in the digest.
- Show that a small input change produces a large digest difference.

3. Search for two different short messages that result in the same SHA-1 digest

Signature:

Date:

Program-8: Secure Bitcoin Transaction Signing with Elliptic Curve Cryptography

Algorithm:

Step	Description	MATLAB Function / Code Used
1	**Initialize Elliptic Curve Parameters**- Select a small prime `p`.- Define curve parameters `a` and `b`.- Choose a base point `G = (Gx, Gy)`.- Define `n = p` (simplified).	`p = 233; a = 1; b = 1;` `G = [4,5]; n = 233;`
2	**Key Generation**- Generate private key `d` randomly.- Compute public key `Q = d * G` using scalar multiplication.	`d = randi([1 n-1]);` `Q = scalarMult(d, G, a, p);`

3	**Transaction Hashing**- Convert transaction message `Tx` into a numeric hash.- Compute hash: `h = sum(ASCII(Tx_i) * i) mod n.`	`h = 0; for i = 1:length(Tx)`
		`h = mod(h + double(Tx(i)) * i, n);`
		`end`

4	**Signature Generation**- Select random ephemeral key `k`.- Compute `R = k * G`.- Compute signature components: `r = R_x mod n`, `s = k^-1 * (h + d*r) mod n`.- Signature = `(r, s).`	`k = randi([1 n-1]);`
		`R = scalarMult(k, G, a, p);`
		`r = mod(R(1), n);`
		`s = mod(invmod(k,n)*(h+d*r), n);`

5	**Verification (Simple Hash Check)**- Enter message to verify.- Compute hash `h2` for entered message.- Compare `h2` with original `h`.- If equal → VALID; else → INVALID.	`msg = input('Message: ','s');`
		`h2=0;`
		`for i=1:length(msg)`
		`h2 = mod(h2 + double(msg(i))*i,n);`
		`end;`
		`if h2==h disp('VALID'); else`
		`disp('INVALID'); end`

6	**Helper Functions**- `invmod(x,m)` → Modular inverse of `x mod m`.- `pointAdd(P,Q,a,p)` → Elliptic curve point addition.- `scalarMult(k,P,a,p)` → Scalar multiplication using double-and-add.	`function y = invmod(x,m) ...`
		`endfunction`
		`R = pointAdd(P,Q,a,p) ...`
		`endfunction`
		`R = scalarMult(k,P,a,p) ... end`

MATLAB CODE

```
% ECC_bitcoin_simple_verify.m
clc; clear;
disp('=== Simple Bitcoin Transaction Signing & Verification (Hash Match Only)
===');

% --- Curve (toy demo) ---
p = 233; a = 1; b = 1; G = [4,5]; n = 233;

% --- Key generation ---
```

```
d = randi([1 n-1]);           % private key
Q = scalarMult(d, G, a, p);   % public key

% --- Transaction input & simple hash (sum ASCII*position mod n) ---
Tx = input('Enter Bitcoin Transaction Data to SIGN: ','s');
h = 0;
for i = 1:length(Tx)
    h = mod(h + double(Tx(i)) * i, n);
end
fprintf('\nTransaction Hash (h) = %d\n', h);

% --- Single signature generation (one attempt) ---
k = randi([1 n-1]);
R = scalarMult(k, G, a, p);
if isempty(R)
    error('R is point at infinity. Re-run the script.');
end
r = mod(R(1), n);
s = mod(invmod(k, n) * (h + mod(d*r, n)), n);
fprintf('Generated Signature: (r=%d, s=%d)\n', r, s);

% ==============================
% --- Simple Verification ---
% ==============================
msg = input('\nEnter message to VERIFY: ','s');
h2 = 0;
for i = 1:length(msg)
    h2 = mod(h2 + double(msg(i)) * i, n);
end
fprintf('Computed Hash from entered message = %d\n', h2);

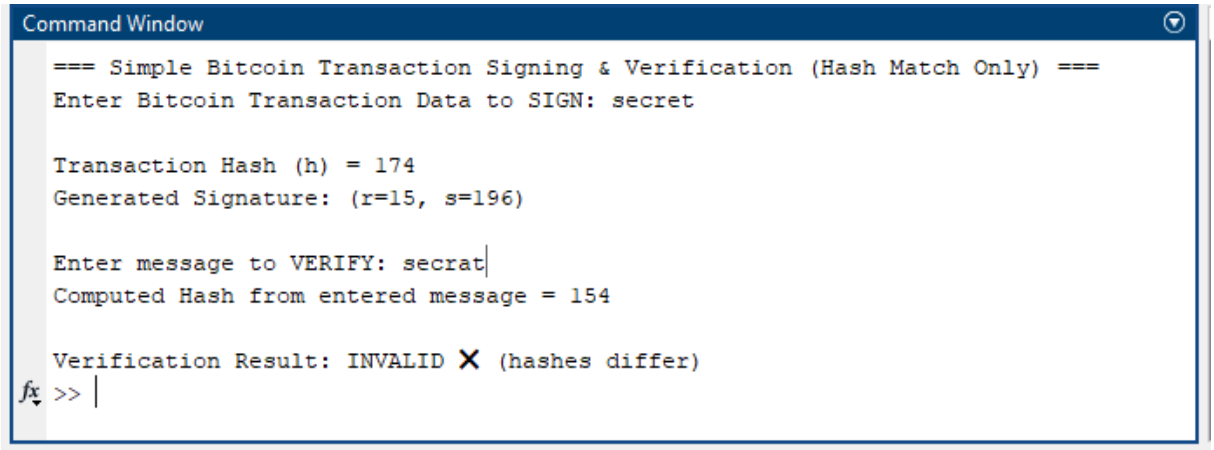
if h2 == h
    fprintf('\nVerification Result: VALID ✓ (hashes match)\n');
else
    fprintf('\nVerification Result: INVALID ✗ (hashes differ)\n');
end

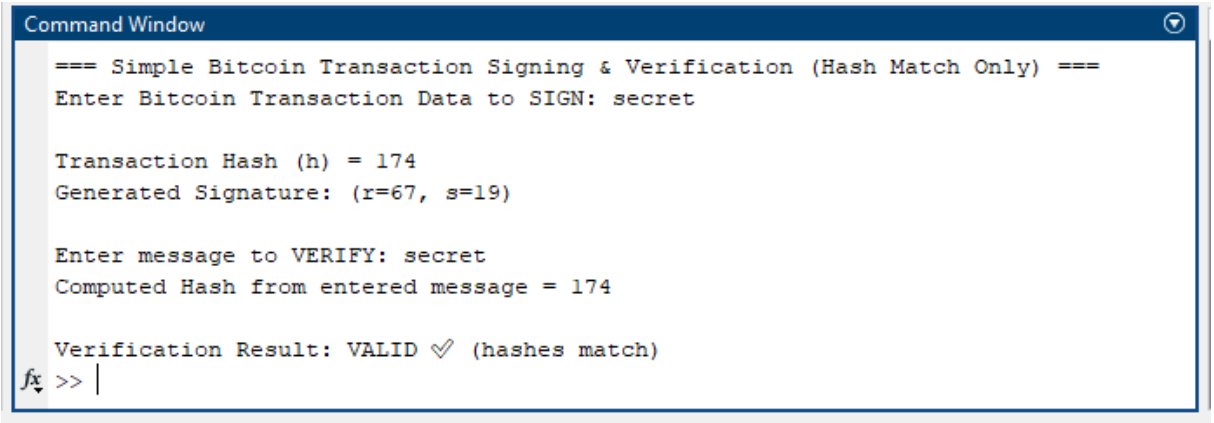
%% ---- Helper functions ----
function y = invmod(x, m)
    x = mod(x, m);
    [g, u, ~] = gcd(x, m);
    if g ~= 1
        error('Modular inverse does not exist.');
    end
    y = mod(u, m);
end

function R = pointAdd(P, Q, a, p)
    if isempty(P), R = Q; return; end
    if isempty(Q), R = P; return; end
    x1 = P(1); y1 = P(2); x2 = Q(1); y2 = Q(2);
    if x1 == x2 && mod(y1 + y2, p) == 0
        R = []; return;
    end
    if x1 == x2 && y1 == y2
        den = mod(2*y1, p);
        if den == 0, R = []; return; end
        lam = mod((3*x1^2 + a) * invmod(den, p), p);
    else
        den = mod(x2 - x1, p);
```

```
        if den == 0, R = []; return; end
        lam = mod((y2 - y1) * invmod(den, p), p);
    end
    xr = mod(lam^2 - x1 - x2, p);
    yr = mod(lam * (x1 - xr) - y1, p);
    R = [xr, yr];
end

function R = scalarMult(k, P, a, p)
    R = [];
    if isempty(P) || k == 0, return; end
    Q = P;
    while k > 0
        if bitand(k, 1)
            R = pointAdd(R, Q, a, p);
        end
        Q = pointAdd(Q, Q, a, p);
        k = floor(k/2);
    end
end
```





Assignment: The current program generates a **hash-based signature** and verifies messages by hash matching. Extend this program to include **new features**

1. Multiple Message Signing:

- Modify the program to allow the user to sign **multiple messages** in a single run.
- Store each message, its hash, and signature `(r, s)` in arrays or a table.
- Display all signed messages and signatures at the end.

- **Signature Verification for Multiple Messages:**

  - Allow the user to verify **any previously signed message** by selecting it from a list.
  - Show whether the signature is **VALID** or **INVALID**.

4. **Tampering Check:**

- Let the user **enter a different message** for verification to simulate tampering.
- Show that verification fails if the message or signature is changed.

Signature:

Date:

Program-9: Implementing Digital Signatures for Secure E-Governance Services (Tax Filing, Aadhaar e-KYC, and E Voting)

Algorithm: DSA-Style Digital Signature Generation

Input: A message `msg` to be signed
Output: Digital signature `(r, s)` in hexadecimal format

Step 1: Initialize DSA Parameters

1. Choose a small prime `p` (modulus).
2. Choose a subgroup order `q` (prime divisor of `p-1`).
3. Choose a generator `g` of the subgroup of order `q`.

Step 2: Generate Keys

1. Select a random **private key** `x` in the range `[1, q-1]`.
2. Compute the **public key**: `y = g^x mod p`.

Step 3: Hash the Message

1. Convert the message into a **numeric hash** `h`.
   - For this demo, use a simple hash:
     `h = mod(sum(position * ASCII code), q)`
     where `position` is the character index in the message.

Step 4: Generate the Signature

1. Repeat until valid signature is found:
   a. Select a random **ephemeral key** `k` in `[1, q-1]`.
   b. Compute `r = (g^k mod p) mod q`. If `r = 0`, choose a new `k`.
   c. Compute `s = k^-1 * (h + x*r) mod q`. If `s = 0`, choose a new `k`.
2. Once both `r` and `s` are non-zero, the signature `(r, s)` is valid.

Step 5: Convert Signature to Hexadecimal (Optional)

1. Convert `r` to hexadecimal `r_hex`.
2. Convert `s` to hexadecimal `s_hex`.
3. Concatenate `r_hex` and `s_hex` to form the **signature string** for display or transmission.

Step 6: Output

- Display the signature in hex: `signatureHexStr = r_hex || s_hex`.

Helper Functions Used:

1. `modinv(a, m)` — Computes modular inverse of `a` modulo `m`.
2. `powermod(a, b, m)` — Computes `(a^b) mod m` using **double-and-add method**.

Notes:

- This version uses small primes for demonstration and teaching purposes.
- The message hash is simplified; in real DSA, a cryptographic hash (e.g., SHA-256) is used.
- `(r, s)` is the actual digital signature; converting to hex is optional for readability.

MATLAB Code:

```
clc; clear;
disp('=== DSA-style Digital Signature (Pure MATLAB, Hex Output) ===');

% --- DSA Parameters (small primes for demo) ---
p = 467; q = 233; g = 2;

% --- Private and Public Key ---
x = randi([1 q-1]);          % private key
y = powermod(g, x, p);       % public key

% --- Message and simple hash ---
msg = input('Enter message to sign: ','s');
h = mod(sum((1:length(msg)).*double(msg)), q);  % simple hash

% --- Signature generation ---
while true
    k = randi([1 q-1]);
    r = mod(powermod(g, k, p), q);
    if r==0, continue; end
    s = mod(modinv(k, q)*(h + x*r), q);
    if s ~=0, break; end
end

% --- Convert signature to hex string ---

r_bytes = dec2hex(r, 2);
s_bytes = dec2hex(s, 2);
```

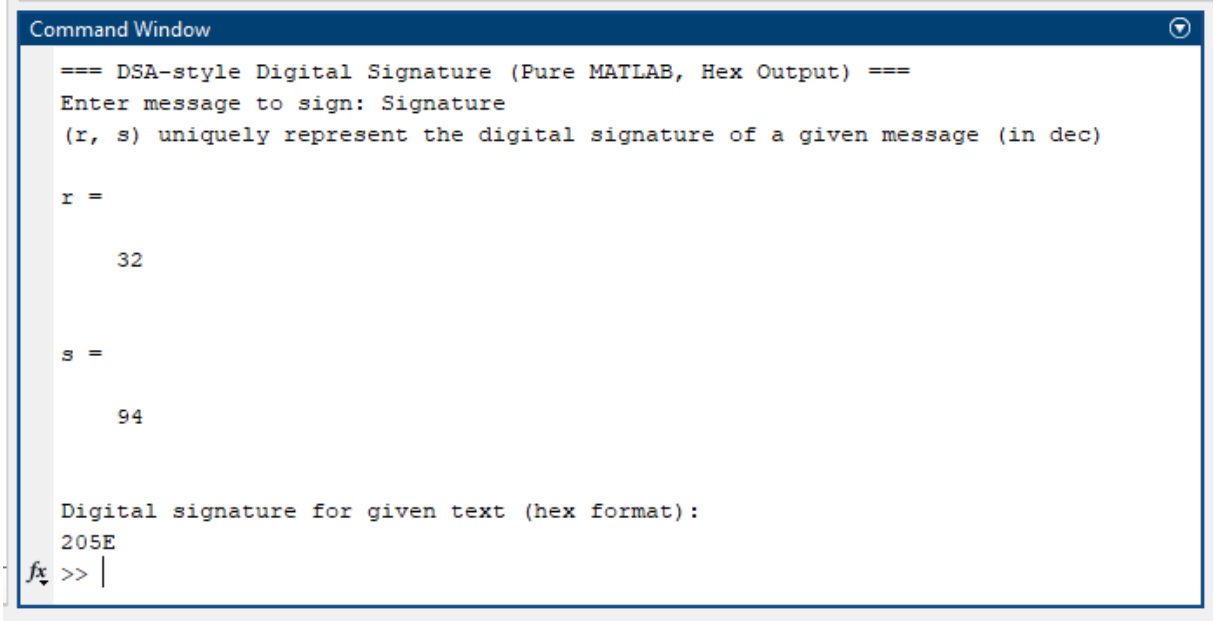
```
disp('(r, s) uniquely represent the digital signature of a given message (in
dec) ')
r
s

signatureHexStr = [r_bytes s_bytes];  % concatenate hex of r and s

fprintf('\nDigital signature for given text (hex format):\n%s\n',
signatureHexStr);

%% --- Helper functions ---
function x = modinv(a,m)
[g,u,~] = gcd(a,m);
if g ~= 1, error('No modular inverse exists'); end
x = mod(u,m);
end

function y = powermod(a,b,m)
y = 1; a = mod(a,m);
while b>0
    if bitand(b,1), y = mod(y*a,m); end
    a = mod(a*a,m); b=floor(b/2);
end
end
```



Assignment:

1. Write a MATLAB program that takes as input:
   - The **message** `msg`.
   - The **generated signature** `(r, s)`.
   - The **public key** `y`.
2. Compute the **hash of the message** `h` using the same hash function as in the signature generation (e.g., `h = mod(sum(position × ASCII code), q)`).
3. Implement the **DSA signature verification steps**:

$$w = s^{-1} \mod q$$

$$u_1 = h \cdot w \mod q$$

$$u_2 = r \cdot w \mod q$$

$$v = \left(g^{u_1} \cdot y^{u_2} \mod p\right) \mod q$$

4. **Compare** `v` with `r`.
   - If `v = r`, the signature is **VALID**.
   - Otherwise, the signature is **INVALID**.

HINT: check workspace variables.

Signature:

Date:

Program 10: a. Build a Trojan and know the impact of the Trojan malwares in a computer system b. Install a rootkit hunter and find the malwares in a computer

PART A: Build a Trojan and know the impact of the Trojan malwares in a computer system

PROCEDURE:

1. Create a simple trojan by using Windows Batch File (.bat)
2. Type these below code in notepad and save it as Trojan.bat
3. Double click on Trojan.bat file.
4. When the trojan code executes, it will open MS-Paint, Notepad, Command Prompt, Explorer, etc., infinitely.
5. Restart the computer to stop the execution of this trojan.

TROJAN:

- In computing, a **Trojan horse**, or simply a **trojan**, is a type of malware that deceives users about its true purpose.
- Trojans typically spread through **social engineering techniques**, such as tricking a user into opening an email attachment that appears harmless (e.g., a routine form) or clicking on a fake advertisement on social media or other platforms.
- While their effects can vary, many modern trojans function as **backdoors**, allowing a remote controller unauthorized access to the infected system.
- They can enable attackers to steal sensitive personal data, including banking details, passwords, and identity information.
- **Example:** Many ransomware attacks are delivered via trojans.

CODE:

Trojan.bat

```
@echo off
:x
start mspaint
start notepad
start cmd
start explorer
start control
start calc
goto x
```

OUTPUT:

(MS-Paint, Notepad, Command Prompt, Explorer will open infinitely)

PART B: Install a rootkit hunter and find the malwares in a computer

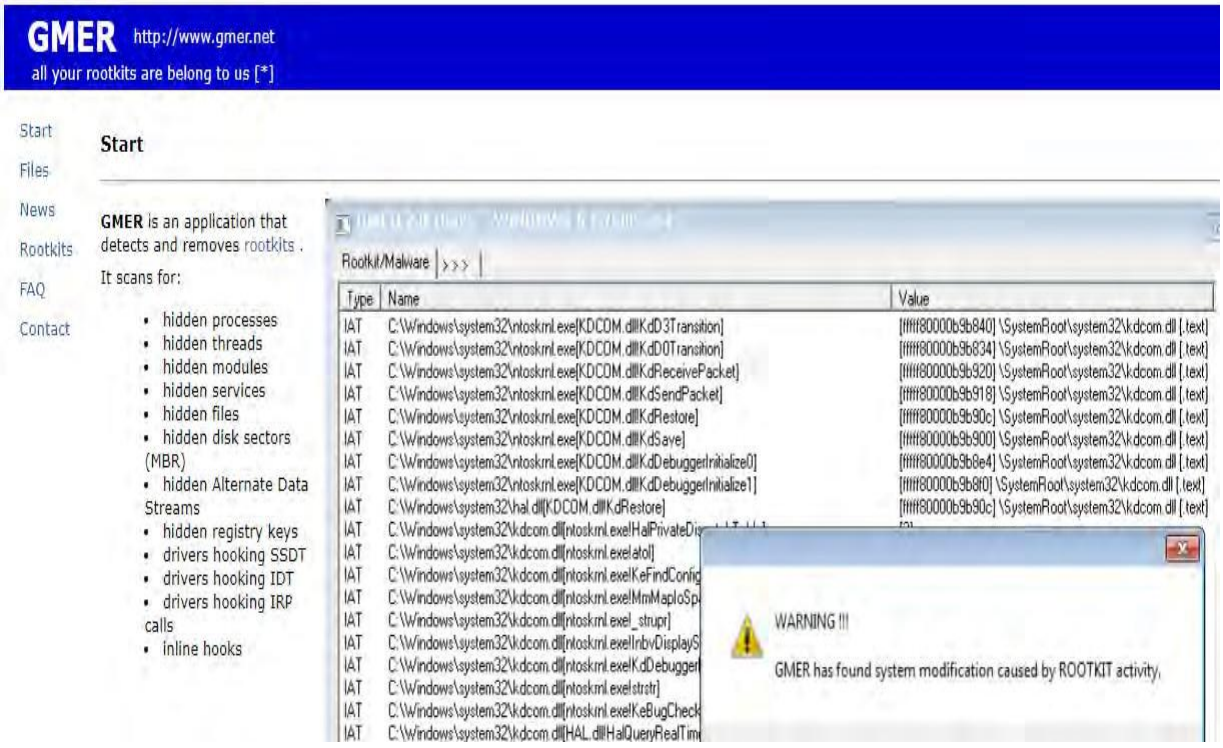
ROOTKIT HUNTER:

- **rkhunter (Rootkit Hunter)** is a Unix-based utility designed to scan systems for rootkits, backdoors, and potential local exploits.
- It works by **comparing SHA-1 hashes** of critical files against known good values from online databases, checking for default rootkit directories, incorrect file permissions, hidden files, and suspicious kernel module strings, and performing specialized tests for Linux and FreeBSD.
- rkhunter is widely recognized because it is included in popular Linux distributions such as **Fedora** and **Debian**.
- The tool is written in **Bourne shell**, making it highly portable and capable of running on most UNIX-derived systems.

GMER ROOTKIT TOOL:

- **GMER** is a rootkit detection and removal tool developed by Polish researcher **Przemysław Gmerek**.
- It operates on **Microsoft Windows** and supports versions including NT, 2000, XP, Vista, 7, 8, and 10.
- Starting from version **2.0.18327**, GMER provides full support for **Windows x64 systems**.

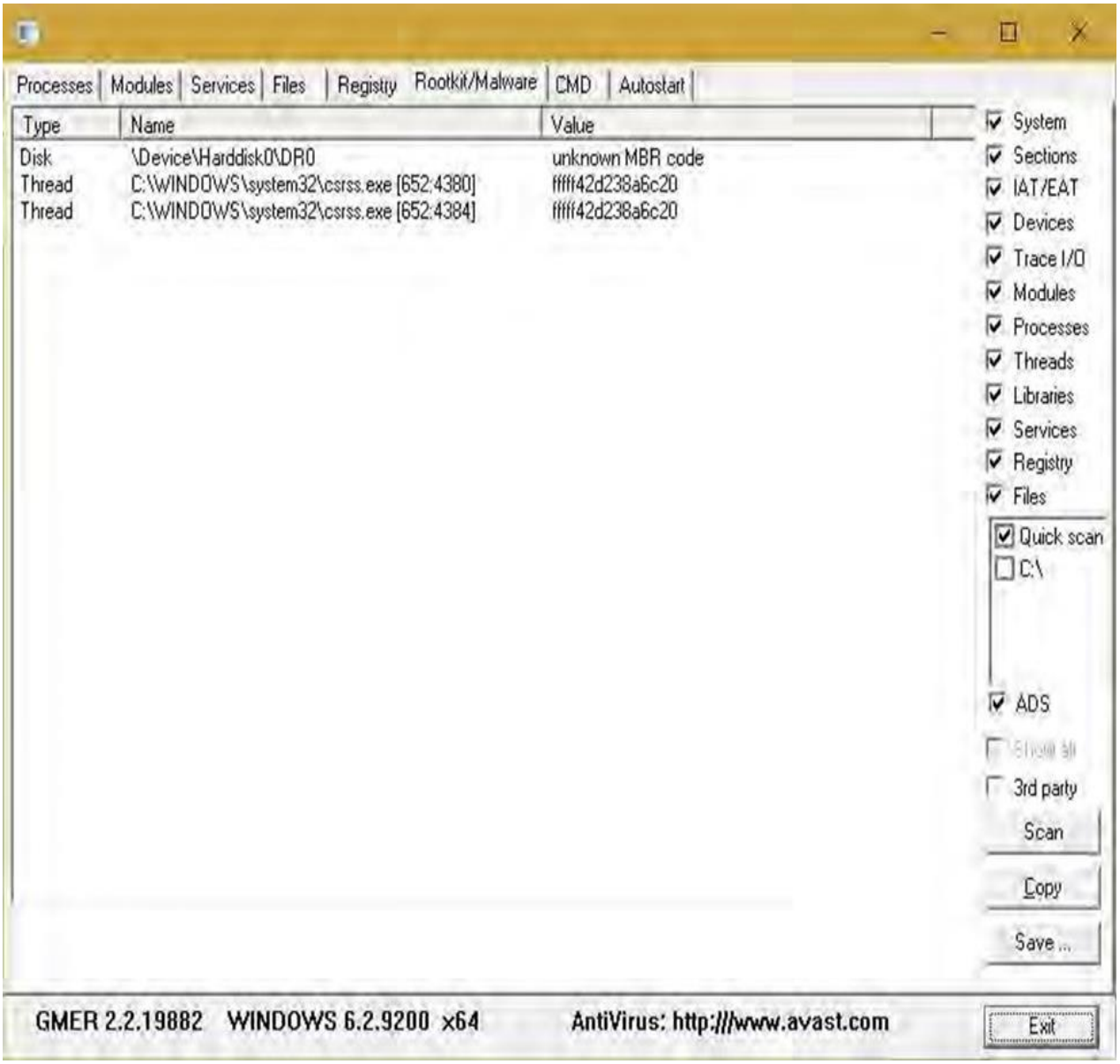
Step 1



Visit GMER's website (see Resources) and download the GMER executable.

Click the "Download EXE" button to download the program with a random file name, as some rootkits will close “gmer.exe” before you can open it.

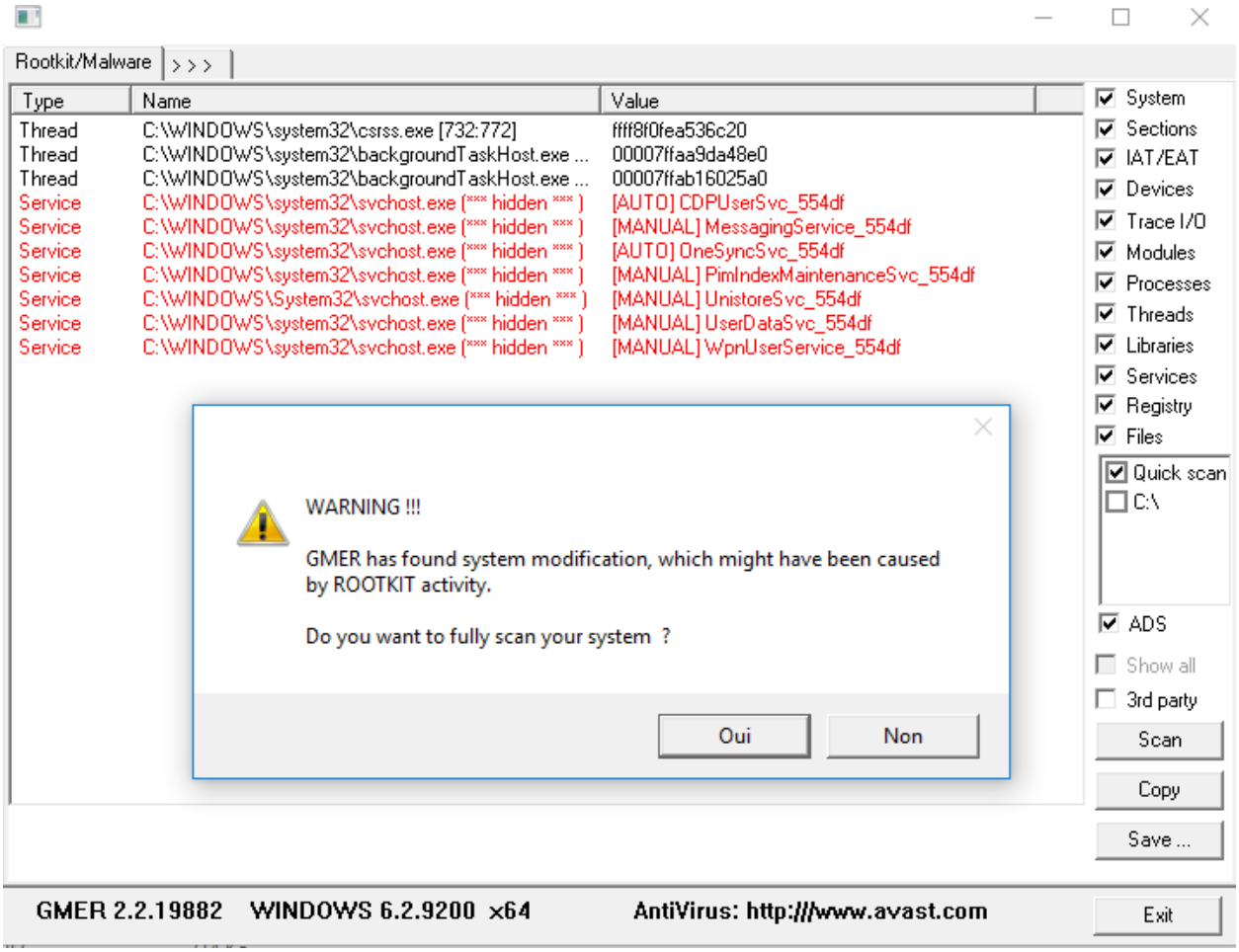
Step 2



Double-click the icon for the program.

Click the "Scan" button in the lower-right corner of the dialog box. Allow the program to scan your entire hard drive.

Step 3



When the program completes its scan, select any program or file listed in red. Right-click it and select "Delete."

If the red item is a service, it may be protected. Right-click the service and select "Disable." Reboot your computer and run the scan again, this time selecting "Delete" when that service is detected.

When your computer is free of Rootkits, close the program and restart your PC.

RESULT:

In this experiment a rootkit hunter software tool has been installed and the rootkits have been detected.

Signature:

Date:

Program 11: Induce and combat an attack by intrusion detection system (IDS) using any network software tools.

Aim: To demonstrate Intrusion Detection System (IDS) using Snort software tool.

STEPS ON CONFIGURING AND INTRUSION DETECTION:

1. Download Snort from the Snort.org website. (http://www.snort.org/snort- downloads)

2. Download Rules(https://www.snort.org/snort-rules). You must register to get the rules. (You should download these often)

3. Double click on the .exe to install snort. This will install snort in the "C:\Snort" folder.It is important to have WinPcap (https://www.winpcap.org/install/) installed

4. Extract the Rules file. You will need WinRAR for the .gz file.

5. Copy all files from the "rules" folder of the extracted folder. Now paste the rules into *"C:\Snort\rules"* folder.

6. Copy "snort.conf" file from the "etc" folder of the extracted folder. You must paste it into "C:\Snort\etc" folder. Overwrite any existing file. Remember if you modify your snort.conf file and download a new file, you must modify it for Snort to work.

7. Open a command prompt (cd.exe) and navigate to folder "C:\Snort\bin" folder. (at the Prompt, type cd\snort\bin)

8. To start (execute) snort in sniffer mode use following command: snort -dev -i 3

-i indicates the interface number. You must pick the correct interface number. In my case, it is 3.

-dev is used to run snort to capture packets on your network.

To check the interface list, use following command: snort -W



Finding an interface

You can tell which interface to use by looking at the Index number and finding Microsoft. As you can see in the above example, the other interfaces are for VMWare. My interface is 3.

9. To run snort in IDS mode, you will need to configure the file "snort.conf" according to your network environment.

10. To specify the network address that you want to protect in snort.conf file, look for the following line.

var HOME_NET 192.168.1.0/24 (You will normally see any here)

11. You may also want to set the addresses of DNS_SERVERS, if you have some on your network.

Example:

example snort

12. Change the RULE_PATH variable to the path of rules folder. var RULE_PATH c:\snort\rules

path to rules

13. Change the path of all library files with the name and path on your system. and you must change the path of snort_dynamicpreprocessorvariable. C:\Snort\lib\snort_dynamiccpreprocessor

You need to do this to all library files in the "C:\Snort\lib" folder. The old path might be: "/usr/local/lib/…". you will need to replace that path with your system path. Using C:\Snort\lib

14. Change the path of the "dynamicengine" variable value in the "snort.conf" file..

Example:

dynamicengine C:\Snort\lib\snort_dynamicengine\sf_engine.dll

15 Add the paths for "include classification.config" and "include reference.config" files.

include c:\snort\etc\classification.config

include c:\snort\etc\reference.config

16. Remove the comment (#) on the line to allow ICMP rules, if it is commented with a #.

include $RULE_PATH/icmp.rules

17. You can also remove the comment of ICMP-info rules comment, if it is commented.

include $RULE_PATH/icmp-info.rules

18. To add log files to store alerts generated by snort, search for the "output log" test in snort.conf and add the following line:

output alert_fast: snort-alerts.ids

19. Comment (add a #) the whitelist $WHITE_LIST_PATH/white_list.rules and the blacklist Change the nested_ip inner , \ to nested_ip inner #, \

20. Comment out (#) following lines:
#preprocessor normalize_ip4
#preprocessor normalize_tcp: ips ecn stream
#preprocessor normalize_icmp4 #preprocessor normalize_ip6
#preprocessor normalize_icmp6

21. Save the "snort.conf" file.

22. To start snort in IDS mode, run the following command:

snort -c c:\snort\etc\snort.conf -l c:\snort\log -i 3 (Note: 3 is used for my interface card)

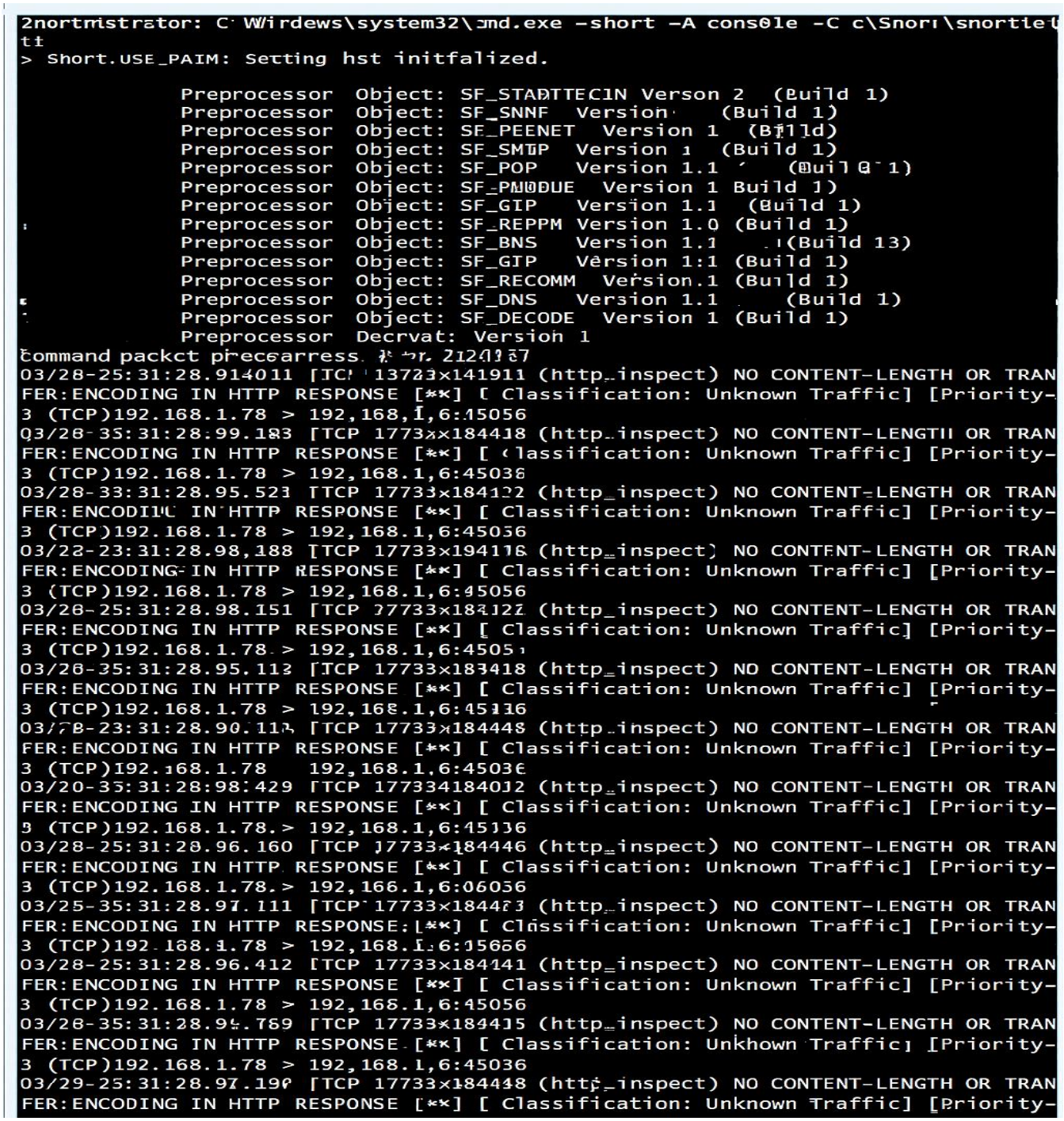
If a log is created, select the appropriate program to open it. You can use WordPard or NotePad++ to read the file.

To generate Log files in ASCII mode, you can use following command while running snort in IDS mode:
snort -A console -i3 -c c:\Snort\etc\snort.conf -l c:\Snort\log -K ascii

23. Scan the computer that is running snort from another computer by using PING or NMap (ZenMap).

After scanning or during the scan you can check the snort-alerts.ids file in the log folder to insure it is logging properly. You will see IP address folders appear.

Snort monitoring traffic –



RESULT:

Thus the Intrusion Detection System(IDS) has been demonstrated by using the Open Source Snort Intrusion Detection Tool.

Signature:

Date:

Program 12: Evaluate vulnerability of a network by automated Attack and Penetration Tools

AIM:

To download the N-Stalker Vulnerability Assessment Tool and exploring the features.

EXPLORING N-STALKER:

- N-Stalker Web Application Security Scanner is a Web security assessment tool.
- It incorporates with a well-known N-Stealth HTTP Security Scanner and 35,000 Web attack signature database.
- This tool also comes in both free and paid version.
- Before scanning the target, go to "License Manager" tab, perform the update.
- Once update, you will note the status as up to date.
- You need to download and install N-Stalker from www.nstalker.com.

1. Start N-Stalker from a Windows computer. The program is installed under Start ⇨ Programs ⇨ N-Stalker ⇨ N-Stalker Free Edition.
2. Enter a host address or a range of addresses to scan.
3. Click Start Scan.
4. After the scan completes, the N-Stalker Report Manager will prompt you to select a format for the resulting report as choose Generate HTML.
5. Review the HTML report for vulnerabilities.





Now goto “Scan Session”, enter the target URL.
In scan policy, you can select from the four options,

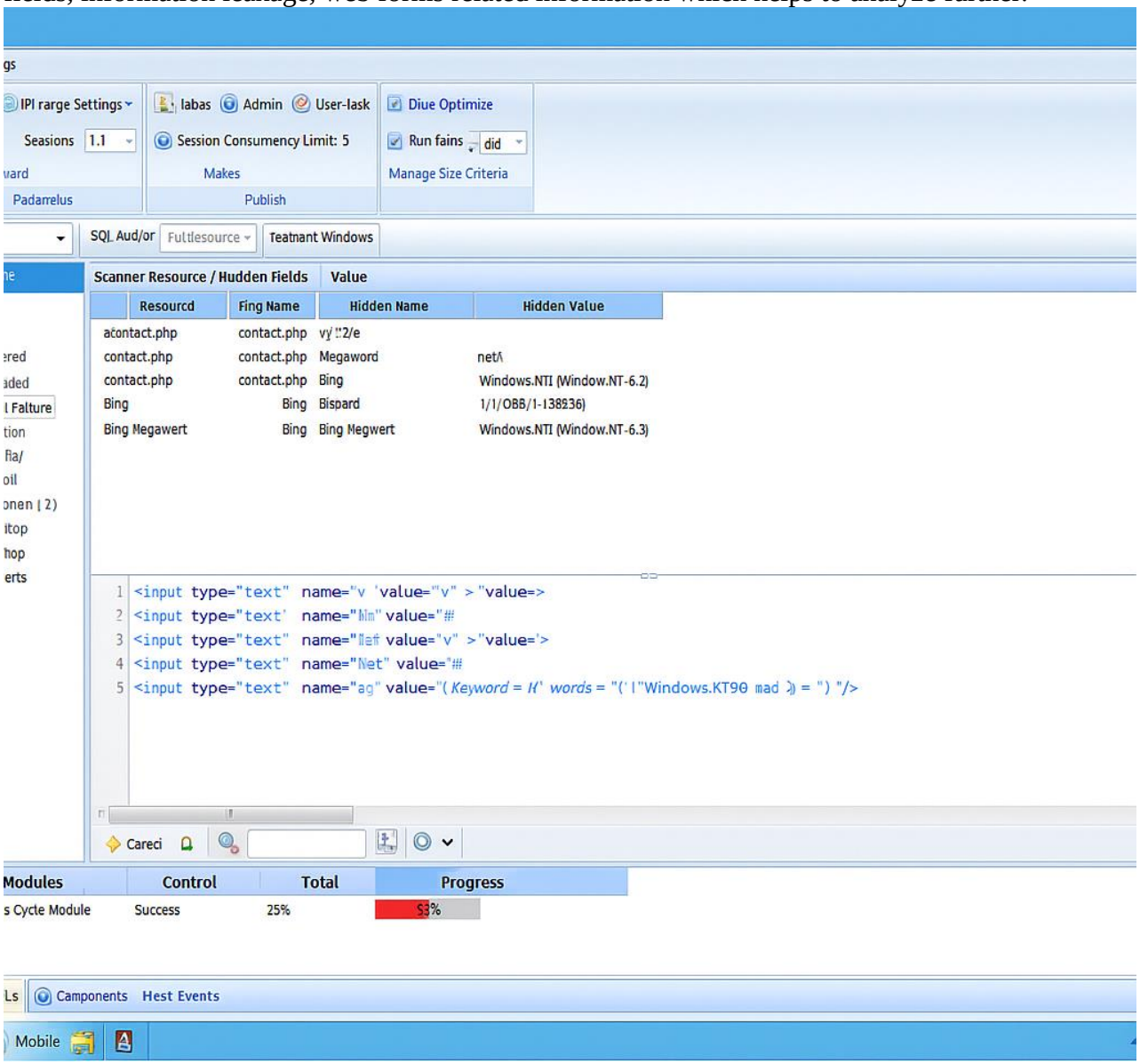
- Manual test which will crawl the website and will be waiting for manual attacks.
- full xss assessment
- owasp policy
- Web server infrastructure analysis.

Once, the option has been selected, next step is “Optimize settings” which will crawl the whole website for further analysis.
In review option, you can get all the information like host information, technologies used, policy name, etc.

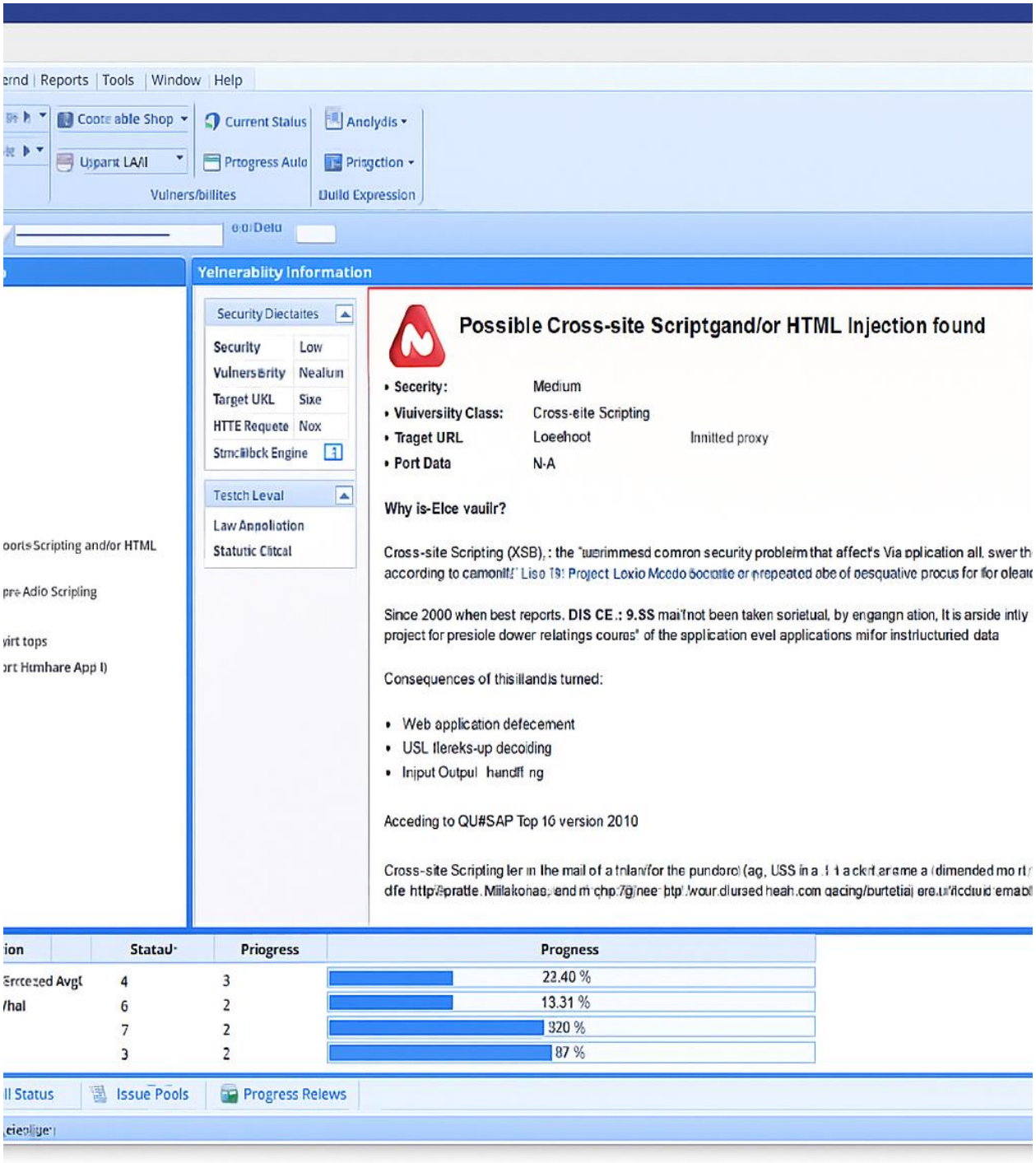




Once done, start the session and start the scan.
The scanner will crawl the whole website and will show the scripts, broken pages, hidden fields, information leakage, web forms related information which helps to analyze further.



Once the scan is completed, the NStalker scanner will show details like severity level, vulnerability class, why is it an issue, the fix for the issue and the URL which is vulnerable to the particular vulnerability?



RESULT:

Thus the N-Stalker Vulnerability Assessment tool has been downloaded, installed and the features has been explored by using a vulnerable website.

Signature:

REFERENCES

- **WikiPedia**
  - https://en.wikipedia.org/wiki/GMER
  - https://en.wikipedia.org/wiki/Snort_(software)
  - https://en.wikipedia.org/wiki/Trojan_horse_(computing)

- **GeeksForGeeks**
  - https://www.geeksforgeeks.org/rsa-algorithm-cryptography/
  - https://www.geeksforgeeks.org/sha-1-hash-in-java/
  - https://www.geeksforgeeks.org/caesar-cipher-in-cryptography/
  - https://www.geeksforgeeks.org/how-to-solve-rsa-algorithm-problems/
  - https://www.geeksforgeeks.org/playfair-cipher-with-examples/

- **Build Your Own Security Lab, Michael Gregg, Wiley India**
  - https://doc.lagout.org/security/Build%20Your%20Own%20Security% 20Lab%20for%20Network%20Testing.pdf

- **Some other Websites**
  - https://forums.malwarebytes.com/topic/200301-sneaky-rootkit-help- needed/
  - https://itstillworks.com/use-gmer-remove-rootkit-6102694.html

🔐 Cryptography Question Bank

Basics of Cryptography

1. Define Cryptography and its benefits.
2. What are the major applications of cryptography in the modern world?
3. What type of information can be secured using cryptography?
4. What is plaintext?
5. What is ciphertext?
6. What is decryption and why is it needed?
7. What is encryption and how does it work?
8. Explain the difference between cryptography and steganography.

Types of Cryptography

9. What is Secret Key Cryptography (Symmetric Cryptography)?
10. What is Public Key Cryptography (Asymmetric Cryptography)?
11. Differentiate between symmetric and asymmetric encryption.
12. What is the difference between a private key and a public key?
13. What is hybrid cryptography?

Algorithms and Techniques

14. What is the Caesar Cipher?
15. What is a block cipher?
16. What is a stream cipher?
17. What is a transposition cipher?
18. What is a substitution cipher?
19. What is the International Data Encryption Algorithm (IDEA)?
20. What do you know about RSA encryption algorithm?
21. What are the mathematical algorithms used in symmetric cryptography?
22. What are the mathematical algorithms used in asymmetric cryptography?
23. What is the Digital Signature Algorithm (DSA)?
24. What is Elliptic Curve Cryptography (ECC)?

Infrastructure and Systems

25. What is a Key Distribution Center (KDC) and how is it used?
26. What are the specific components of a Public Key Infrastructure (PKI)?
27. What is a digital certificate?
28. What is the role of a Certificate Authority (CA)?
29. What is the role of a Registration Authority (RA)?

Hashing and Authentication

30. What is a cryptographic hash function?

31. List and describe some common hashing algorithms.
32. How is hashing different from encryption?
33. What is message integrity and how is it ensured?
34. What is a digital signature and how is it verified?

Theoretical and Mathematical Foundations

35. What is **Shannon's concept of perfect secrecy**?
36. Define **confusion and diffusion** in the context of block ciphers.
37. How do **modular arithmetic** and **prime factorization** underpin modern encryption algorithms?
38. Describe the concept of **trapdoor functions** and give examples.
39. Explain **discrete logarithm problem** and its relevance in cryptographic systems.
40. Compare the hardness assumptions behind **RSA**, **Diffie-Hellman**, and **Elliptic Curve Cryptography**.
41. What is a **cryptographic nonce** and why is it important?
42. Explain the concept of **timing attacks** in cryptography.
43. What is **differential cryptanalysis** and how does it affect symmetric ciphers?
44. Describe **linear cryptanalysis** and give an example.
45. How does **replay attack** work and how can cryptographic protocols defend against it?
46. What are the weaknesses of RSA when small key sizes are used?

ASSIGNMENT WORKSHEETS: (insert 15-20 plain sheets before binding)